

Chapter 5

Contents

Chapters

- 5. Programming PIC 18 using Assembly & C Language
 - 5.1 Assembly Language Programming Examples
 - 5.1.1 Basic programming examples
 - 5.1.2 Bank Switching
 - 5.1.3 Macros & Subroutines
 - 5.2 C Language Programming Examples

This chapter presents the programming of PIC18 Microcontroller using Assembly and high-level C language programming.

5.1 Assembly Language Programming Examples

This section presents the programming of PIC18 Microcontroller using Assembly language programming. It is started with Basic programming examples to familiarize with registers, instructions, addressing modes and their usage. It is followed by examples that demonstrate the concept of Banks & switching between banks. Later the concept of modularization & its variation such as Macros & Subroutines are presented

5.1.1 Basic programming examples

Program 5.1 Addition of two 8-Bit numbers using different addressing modes

Program 5.1(a) 8-Bit Addition using Immediate Addressing

The program performs addition of literals 0x34 and 0x23 using immediate addressing. The result 0x57 is stored in RAM location 0x30.

```
#INCLUDE <P18F458.INC> ; Include library for the specific processor
ORG      0x00          ; Set the origin address to 0x00 of Flash
                        ; Reset Vector
GOTO     START         ; Goto /Jump to Start of Program
ORG      0x30          ; Set the origin address to 0x30 of Flash; User Space
START    MOVLW 0x34     ; Move literal value 0x34 to WREG; WREG= 0x34
        ADDLW 0x23     ; Add literal 0x23 with literal 0x34 (in WREG)
                        ; Store the result back in WREG; WREG= 0x57
        MOVWF 0x30     ; Move the addition result stored in WREG to RAM
                        ; location 0x30H, [0x30] = 0x57
        NOP           ; No operation; To use for breakpoint debug.
        END           ; Set end of the program for assembler.
```

Result

```
0x34    ßLiteral 1
0x23   ßLiteral 2
0x57    ßFile Register 0x30
```

Program 5.1(b) 8-Bit Addition using Direct Addressing

The program performs addition of values 0x34 and 0x23 that are actually present in File Registers (RAM locations) 0x10 and 0x20 respectively using direct addressing method. The result 0x57 is stored in file register 0x30.

```
#INCLUDE <P18F458.INC> ; Include library for the specific processor
ORG      0x00          ; Set the origin address to 0x00 of Flash
                        ; Reset Vector
GOTO     START         ; Goto/Jump to Start of Program
```

	ORG	0x30	; Set the origin address to 0x30 of Flash; User Space
START	MOVF	0x10,W	; Move 0x34 present in file register 0x10 to WREG ; WREG= [0x10], WREG= 0x34
	ADDWF	0x20,W	; Add 0x23 present in file register 0x20 with 0x34 ; (in WREG), Store the result back in WREG ; WREG= WREG+ [0x20], WREG= 0x57
	MOVWF	0x30	; Move the addition result stored in WREG to file ; register 0x30; [0x30] = 0x57
	NOP		; No operation; To use for breakpoint debug.
	END		; Set end of the program for assembler.

Result 0x34 ßFile Register 0x10
0x23 ßFile Register 0x20
0x57 ßFile Register 0x30

Program 5.1(c) 8-Bit Addition using Indirect Addressing

The program performs addition of 0x34 and 0x23 that are actually present in file registers 0x110 and 0x120, accessed by pointer FSR0 and FSR1 using indirect/indexed addressing method. The result 0x57 is stored in file register 0x130.

	#INCLUDE <P18F458.INC> ; Include library for the specific processor		
	ORG	0x00	; Set the origin address to 0x00 of Flash ; Reset Vector
	GOTO	START	; Goto/Jump to Start of Program
	ORG	0x30	; Set the origin address to 0x30 of Flash ; User Space
START	LFSR	0,0x110	; Set the pointer 1, FSR0 = 0x110
	LFSR	1,0x120	; Set the pointer 2, FSR1 = 0x120
	LFSR	2,0x130	; Set the pointer 3, FSR2 = 0x130
	MOVF	POSTINC0,0	; Move 0x34 present in file register 0x110 to WREG ; WREG= [0x110], WREG= 0x34, ; FSR0 = FSR0+1, After execution FSR0 = 0x111
	ADDWF	POSTINC1,0	; Add 0x23 present in file register 0x120 with 0x34 ; (in WREG), Store the result back in WREG. ; WREG= WREG+ [0x120], WREG= 0x57 ; FSR1 = FSR1+1, After execution FSR1 = 0x121
	MOVWF	INDF2	; Move the addition result stored in WREG to ; file register 0x130; [0x130] = 0x57 ; After execution FSR2 = 0x130
	NOP		; No operation; To use for breakpoint debug.
	END		; Set end of the program for assembler.

Result 0x34 ßFile Register 0x110 Pointed by FSR0
0x23 ßFile Register 0x120 Pointed by FSR1
0x57 ßFile Register 0x130 Pointed by FSR2

The summary & side by side comparison of the three addressing modes is presented below.

Immediate Addressing		Direct Addressing		Indirect Addressing	
#INCLUDE <P18F458.INC>		#INCLUDE <P18F458.INC>		#INCLUDE <P18F458.INC>	
ORG	0X00	ORG	0X00	ORG	0X00
GOTO	START	GOTO	START	GOTO	START
START ORG	0x30	START ORG	0x30	START ORG	0x30
MOVLW	0X34	MOVF	0X10,W	LFSR	0,0X110
ADDLW	0X23	ADDWF	0X20,W	LFSR	1,0X120
MOVWF	0X30	MOVWF	0X30	LFSR	2,0X130
NOP		NOP		MOVF	POSTINC0,0
END		END		ADDWF	POSTINC1,0
				MOVWF	INDF2
				NOP	
				END	

Program 5.1(a)

Program 5.1(b)

Program 5.1(c)

Note: Any Register indirect, indexed, relative addressing mode is considered as indirect addressing mode during these programming examples. Also, in the following examples [0xYZ] indicates a file register where YZ is an 8-Bit address.

The programs presented in this section from program 5.2 present only the actual logic of the program and not the full format that is written in an assembly file for executing the program.

Program 5.2(a) Addition of 16-Bit number using direct addressing mode.

The program performs 16-Bit addition using direct addressing mode. Low 8-Bit addition of 0x34 and 0x23 that are actually present in file registers 0x10 and 0x20, accessed by pointer FSR0 and FSR1 using indirect/indexed addressing method. The result 0x57 is stored in file register 0x30. High 8-Bit addition of 0x56 and 0x45 that are present in file registers 0x11 and 0x21, accessed by pointer FSR0 and FSR1 using indirect/indexed addressing method. The result 0x9B is stored in file register 0x31.

Direct Addressing		
MOVWF	0x10,W	; Move 0x34 present in file register 0x10 to WREG ; WREG= [0x10], WREG= 0x34
ADDWF	0x20,W	; Add 0x23 present in file register 0x20 with 0x34 ; (in WREG), Store the result back in WREG ; WREG= WREG+ [0x20], WREG= 0x57
MOVWF	0x30	; Move the addition result stored in WREG to file ; register 0x30; [0x30] = 0x57
MOVWF	0x11,0	; Move 0x56 present in file register 0x11 to WREG ; WREG= [0x11], WREG = 0x56
ADDWFC	0x21,0	; Add 0x45 present in file register 0x21 with 0x56 ; (in WREG), Store the result back in WREG ; WREG = WREG + [0x21], WREG = 0x9B
MOVWF	0x31	; Move the addition result stored in WREG to file ; register 0x31; [0x31] = 0x57

Result File Register 0x11à 0x56 0x34 ßFile Register 0x10

<u>File Register 0x21à</u>	<u>0x45</u>	<u>0x23</u>	<u>βFile Register 0x20</u>
File Register 0x31à	0x9B	0x57	βFile Register 0x30

Program 5.2(b) Addition of 16-Bit number using indirect addressing modes.

The program performs 16-Bit addition using indirect addressing mode. Low 8-Bit addition of 0x34 and 0x23 that are present in file registers 0x110 and 0x120, accessed by pointer FSR0 and FSR1 using indirect/indexed addressing method. The result 0x57 is stored in file register 0x130. High 8-Bit addition of 0x56 and 0x45 that are present in file registers 0x111 and 0x121, accessed by pointer FSR0 and FSR1 using indirect/indexed addressing method. The result 0x9B is stored in file register 0x131.

Indirect Addressing

```

LFSR      0,0x110    ; Set the pointer 1, FSR0 = 0x110
LFSR      1,0x120    ; Set the pointer 2, FSR1 = 0x120
LFSR      2,0x130    ; Set the pointer 3, FSR2 = 0x130
MOVF      POSTINC0,0 ; Move 0x34 present in file register 0x110 to WREG
                        ; WREG = [0x110], WREG = 0x34,
                        ; FSR0 = FSR0+1, After execution FSR0 = 0x111
ADDWF     POSTINC1,0 ; Add 0x23 present in file register 0x120 with 0x34
                        ; (in WREG), Store the result back in WREG.
                        ; WREG = WREG + [0x120], WREG = 0x57
                        ; FSR1 = FSR1+1, After execution FSR1 = 0x121
MOVWF     POSTINC2   ; Move the addition result stored in WREG to
                        ; file register 0x130; [0x130] = 0x57
                        ; After execution FSR2 = 0x130
MOVF      POSTINC0,0 ; Move 0x56 present in file register 0x111 to WREG
                        ; WREG = [0x111], WREG = 0x56,
                        ; FSR0 = FSR0+1, After execution FSR0 = 0x112
ADDWFC    POSTINC1,0 ; Add 0x45 present in file register 0x121 with 0x56
                        ; (in WREG), Store the result back in WREG.
                        ; WREG = WREG + [0x121], WREG = 0x9B
                        ; FSR1 = FSR1+1, After execution FSR1 = 0x122
MOVWF     POSTINC2   ; Move the addition result stored in WREG to
                        ; file register 0x131; [0x131] = 0x9B
                        ; After execution FSR2 = 0x131

```

Result

File Register 0x111	à 0x56	0x34	βFile Register 0x110
File Register 0x121	à 0x45	0x23	βFile Register 0x120
File Register 0x131	à 0x9B	0x57	βFile Register 0x130

Program 5.3 Subtraction of two 8-Bit in immediate and 16-Bit numbers in direct mode.

The program performs 8-Bit subtraction using immediate addressing mode. 8-Bit subtraction of 0x34 from 0x23. The result 0xEF is stored in file register 0x30.

Direct Addressing

MOVLW	0x23	; Move literal value 0x34 to WREG; WREG = 0x34
SUBLW	0x34	; Subtract literal 0x34 (in WREG) from literal 0x23 ; Store the result back in WREG; WREG = 0xEF
MOVWF	0x30	; Move the subtraction result stored in WREG to ; file register 0x30H, [0x30] = 0xEF

Result

0x23 ßLiteral 2
0x34 ßLiteral 1
 0xEF ßFile Register 0x30

Indirect Addressing

MOVF	0x10,W	; Move 0x34 present in file register 0x10 to WREG ; WREG = [0x10], WREG = 0x34
SUBWF	0x20,W	; Subtract literal 0x34 (in WREG) from literal 0x23 ; present in file register 0x20, Store the result back ; in WREG, WREG = [0x20] - WREG; WREG = 0xEF ; Store the result back in WREG; WREG = 0xEF
MOVWF	0x30	; Move the subtraction result stored in WREG to ; file register 0x30H, [0x30] = 0xEF
MOVF	0x11,0	; Move 0x56 present in file register 0x11 to WREG ; WREG = [0x11], WREG = 0x56
SUBWFB	0x21,0	; Subtract literal 0x56 (in WREG) with borrow ; from literal 0x45 present in file register 0x21, ; Store the result back in WREG, ; WREG = [0x21] - WREG; ; WREG = 0xEE ; Store the result back in WREG; WREG = 0xEE
MOVWF	0x31	; Move the subtraction result stored in WREG to ; file register 0x31H, [0x31] = 0xEE

Result

File Register 0x121 à 0x45 0x23 ßFile Register 0x120
File Register 0x111 à 0x56 0x34 ßFile Register 0x110
 File Register 0x131 à 0xEE 0xEF ßFile Register 0x130

Program 5.4 Demonstration of flags & their status update

MOVLW	0x01	MOVLW	0x01
ADDLW	0x02	SUBLW	0xFF
MOVWF	0x30	MOVWF	0x30

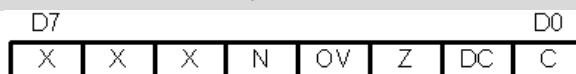


Fig 5.1 Status Register Format

The program with different combinations of values is used to demonstrate the setting and resetting of flags for addition and subtraction instructions. The different combinations are given in Fig 5.2a to Fig 5.2l.

a) $0x01 + 0x02 = 0x03$ à No Flags Set

STATUS	0x00	0	00000000
[STATUS] 4 3 2 1 0			
N OV Z DC C			
0 0 0 0 0			

b) $0x81 + 0x82 = 0x03$ à C, OV Flags Set

STATUS	0x09	9	00001001
[STATUS] 4 3 2 1 0			
N OV Z DC C			
0 1 0 0 1			

c) $0x08 + 0x08 = 0x10$ à DC Flag Set

STATUS	0x02	2	00000010
[STATUS] 4 3 2 1 0			
N OV Z DC C			
0 0 0 1 0			

d) $0xC0 + 0xC0 = 0x80$ à C, Z, OV Flags Set

STATUS	0x11	17	00010001
[STATUS] 4 3 2 1 0			
N OV Z DC C			
1 0 0 0 1			

e) $0x00 + 0x00 = 0x00$ à Z Flag Set

STATUS	0x04	4	00000100
[STATUS] 4 3 2 1 0			
N OV Z DC C			
0 0 1 0 0			

f) $0x80 + 0x80 = 0x00$ à C, Z, OV Flags Set

STATUS	0x0D	13	00001101
[STATUS] 4 3 2 1 0			
N OV Z DC C			
0 1 1 0 1			

g) $0x80 + 0x00 = 0x80$ à N Flag Set

STATUS	0x10	16	00010000
[STATUS] 4 3 2 1 0			
N OV Z DC C			
1 0 0 0 0			

h) $0x7F + 0x01 = 0x80$ à N, OV, DC Flag Set

STATUS	0x1A	26	00011010
[STATUS] 4 3 2 1 0			
N OV Z DC C			
1 1 0 1 0			

i) $0x7F + 0x01 = 0x80$ à N, OV Flag Set

STATUS	0x18	24	00011000
[STATUS] 4 3 2 1 0			
N OV Z DC C			
1 1 0 0 0			

j) $0xFF - 0xFC = 0x03$ à DC, C Flag Set

STATUS	0x03	3	00000011
[STATUS] 4 3 2 1 0			
N OV Z DC C			
0 0 0 1 1			

k) $0x01 - 0xFF = 0x02$ à NO Flags Set

l) $0xFF - 0x01 = 0xFE$ à N, DC, C Flags Set

STATUS	0x00	0	00000000
[STATUS] 4 3 2 1 0 N OV Z DC C 0 0 0 0 0			

STATUS	0x13	19	00010011
[STATUS] 4 3 2 1 0 N OV Z DC C 1 0 0 1 1			

Fig 5.2 Status Flags in different cases.

Program 5.5 Multiplication of two 8-Bit numbers.

The program performs 8-Bit multiplication using indirect addressing mode. 8-Bit multiplication of 0x34 and 0x23 that are actually present in file registers 0x110 and 0x120, accessed by pointer FSR0 and FSR1 using indirect/indexed addressing method. The product result 0x071C is stored in SFR's PRODH (0x07 value) & PRODL (0x1C value) register. The respective values are transferred to WREG one after the other and stored in file register 0x130 & 0x131 accessed by pointer FSR2 using an indirect/indexed addressing method. The result 0x071C is stored in file register 0x131.

LFSR	0,0x110	; Set the pointer 1, FSR0 = 0x110
LFSR	1,0x120	; Set the pointer 2, FSR1 = 0x120
LFSR	2,0x130	; Set the pointer 3, FSR2 = 0x130
MOVF	POSTINC0,0	; Move 0x34 present in file register 0x110 to ; WREG, WREG = [0x110], WREG = 0x34, ; FSR0 = FSR0+1, After execution FSR0 = 0x111
MULWF	POSTINC1,0	; Multiply 0x23 present in file register 0x120 ; with 0x34 (in WREG), Result is stored in ; PRODH, PRODL. ; FSR1 = FSR1+1, After execution FSR1 = 0x121
MOVFF	PRODL,WREG	; Move the lower 8-Bit Production result stored ; in PRODL to WREG, WREG = PRODL
MOVWF	POSTINC2	; Move the low result stored in WREG ; to file register 0x130H, [0x130] = 0x1C ; FSR2 = FSR2+1, After execution FSR2 = 0x131
MOVFF	PRODH,WREG	; Move the higher 8-Bit Production result ; stored in PRODH to WREG, WREG = PRODH
MOVWF	POSTINC2	; Move the higher result stored in WREG ; to file register 0x131H, [0x131] = 0x07 ; FSR2 = FSR2+1, After execution FSR2 = 0x132

Result

		0x23	βFile Register 0x110
		0x34	βFile Register 0x120
File Register 0x131à	0x07	0x1C	βFile Register 0x130

Program 5.6 If Condition

The program performs an equivalence of the following conditional if statement in assembly language.

```
if (x<5)
{
    y=y+1;
    z=z+1;
    x=x+1;
}
```

	MOVF	0x11,0	; WREG = 0x05, i.e [0x11].			
	CPFSLT	0x10	; f = 0x10, [0x10] is x, Skip next instruction			
			; if (f) < (W);			
	BRA	L1	; Branch to L1 if (f) >= (W)			
	INCF	0x13, 1, 0	; Increment File Register 0x13 content à y = y + 1			
	INCF	0x14, 1, 0	; Increment File Register 0x14 content à z = z + 1			
	INCF	0x10, 1, 0	; Increment File Register 0x10 content à x = x + 1			
L1	NOP		; No Operation			

Result

Label	Opcode	Operand	File Register	x value < 5	x value = 5	x value > 5
	MOVF	0x11,0	WREG = [0x11]	5	5	5
	CPFSLT	0x10	[0x10]	0	5	6
	BRA	L1		No	Yes	Yes
	INCF	0x13, 1, 0	[0x13]	1	0	0
	INCF	0x14, 1, 0	[0x14]	1	0	0
	INCF	0x10, 1, 0	[0x10]	1	0	0
L1	NOP					

Program 5.7 While Condition

The program performs an equivalence of the following while loop in assembly language.

```
while (x<5)
{
    y=y+1;
    z=z+1;
    x=x+1;
}
```

	MOVF	0x11,0	; WREG = 0x05, i.e [0x11].			
L2	CPFSLT	0x10	; f = 0x10, [0x10] is x, Skip next instruction if (f) < (W);			

	BRA	L1	; Branch to L1 if (f) >= (W)
	INCF	0x13	; Increment File Register 0x13 content à y = y + 1
	INCF	0x14	; Increment File Register 0x14 content à z = z + 1
	INCF	0x10,1,0	; Increment File Register 0x10 content à x = x + 1
	BRA	L2	; Loop Back to Comparison to make a While
L1	NOP		; No Operation

Result

Label	Op Code	Operand	File Register	Iteration					
				1	2	3	4	5	6
L2	MOVF	0x11,0	WREG = [0x11]	0x05					
	CPFSLT	0x10	x = [0x10]	0x00	0x01	0x02	0x03	0x04	0x05
	BRA	L1		No	No	No	No	No	Yes
	INCF	0x13	[0x13]	0x01	0x02	0x03	0x04	0x05	
	INCF	0x14	[0x14]	0x01	0x02	0x03	0x04	0x05	
	INCF	0x10,1,0	x = [0x10]	0x01	0x02	0x03	0x04	0x05	
L1	BRA	L2		Yes	Yes	Yes	Yes	Yes	No
	NOP			No	No	No	No	No	Yes

Program 5.8 Multi-Byte addition (32-Bit Addition: Four 8-Bit Additions)

The program performs the addition of multiple bytes. It shows the addition of two 32-Bit numbers or 4 bytes.

	LFSR	0,0x110	; Set the pointer 1, FSR0 = 0x110
	LFSR	1,0x120	; Set the pointer 2, FSR1 = 0x120
	LFSR	2,0x130	; Set the pointer 3, FSR2 = 0x130
	MOVLW	0x04	; Loop Count, WREG = 0x04
	MOVWF	0x50	; File register 0x50 = 0x04
	MOVWF	0x51	; File register 0x51 = 0x04
L1	MOVF	POSTINC0,0,0	; Move a byte present in file register starting ; from 0x110 to WREG, WREG = [0x110], ; The file register and respective value changes ; corresponding to the loop iteration. After ; each iteration FSR0 is incremented.
	ADDWFC	POSTINC1,0,0	; Add a byte present in file register starting ; from 0x120 with value in WREG taken from ; 0x110 and so on. The file register and ; respective value changes corresponding to ; the loop iteration. After each iteration FSR0 ; is incremented.

MOVWF	POSTINC2,0	; Move each addition result to file ; register starting from 0x130. After each ; iteration FSR0 is incremented.
DECF	0x51,F,0	; Decrement file register 0x51 by 1 every ; iteration.
BNZ	L1	; Branch to L1 if zero flag is not set. Go to End ; if Zero.

Result

Label	Op Code	Operand	File Register	Iteration			
				1	2	3	4
L1	LFSR	0,0x110	FSR0 = 0x110	0x110			
	LFSR	1,0x120	FSR1 = 0x120	0x120			
	LFSR	2,0x130	FSR2 = 0x130	0x130			
	MOVLW	0x04	WREG = 0x04	0x04			
	MOVWF	0x50	[0x50] = 0x04	0x04			
	MOVWF	0x51	[0x51] = 0x04	0x04			
	MOVF	POSTINC0,0,0	WREG = [POSTINC0]	0x12	0x23	0x34	0x45
	ADDWFC	POSTINC1,0,0	WREG = [POSTINC1]+	+11 = 0x33	+22= 0x45	+33 = 0x67	+44= 0x89
	MOVWF	POSTINC2,0	WREG [POSTINC2] = WREG	0x33	0x45	0x67	0x89
	DECF	0x51,F,0		3	2	1	0
	BNZ	L1		Yes	Yes	Yes	No

POSTINC0 Highest Byte (0x113) à 0x45 0x34 0x23 0x12β Lowest Byte (0x110)
 POSTINC1 Highest Byte (0x123) à 0x44 0x33 0x22 0x11β Lowest Byte (0x120)
 POSTINC2 Highest Byte (0x133) à 0x89 0x67 0x45 0x33β Lowest Byte (0x130)

The same program could be used to perform 8-Bit addition up to 128-Bit addition just by changing the loop count value (Instruction 4) from 1 to 0x10 (16d). This can be extended even further to over 128 bits if the FSR pointers are separated further apart beyond 16 locations.

Program 5.9 Block Data Transfer

The program performs multiple data transfers, or the data transfer of a block. This is a modified version of a multi-byte addition program where FSR1 is not used, and addition is not performed. Instead, data pointed by FSR0 is transferred to locations pointed by FSR2.

Here, the ALP is coded to transfer 32-Bits or 4 bytes from file registers (pointed by FSR0) 0x110, 0x111, 0x112, 0x113 to file registers (pointed by FSR2) 0x120, 0x121, 0x122, 0x123.

	LFSR	0,0x110	; Set the pointer 1, FSR0 = 0x110
	LFSR	2,0x120	; Set the pointer 2, FSR1 = 0x120
	MOVLW	0x04	; Loop Count, WREG = 0x04
	MOVWF	0x50	; File register 0x50 = 0x04
	MOVWF	0x51	; File register 0x51 = 0x04
L1	MOVF	POSTINC0,0	; Move a byte present in file register starting ; from 0x110 to WREG, WREG = [0x110], ; The file register and respective value changes ; corresponding to the loop iteration. After ; each iteration FSR0 is incremented.
	MOVWF	POSTINC2,0	; Move a byte from WREG to file register ; starting from 0x120. The file register changes ; corresponding to the loop iteration. After ; each iteration FSR2 is incremented.
	DECF	0x51,F	; Decrement file register 0x51 by 1 every ; iteration.
	BNZ	L1	; Branch to L1 if zero flag is not set. Go to End ; if Zero.

Result

Label	Op Code	Operand	File Register	Iteration			
				1	2	3	4
L1	LFSR	0,0x110	FSR0 = 0x110	0x110			
	LFSR	2,0x120	FSR2 = 0x120	0x120			
	MOVLW	0x04	WREG = 0x04	0x04			
	MOVWF	0x50	[0x50] = 0x04	0x04			
	MOVWF	0x51	[0x51] = 0x04	0x04			
	MOVF	POSTINC0,0	WREG = [POSTINC0]	0x12	0x23	0x34	0x45
	MOVWF	POSTINC2,0	[POSTINC2] = WREG	0x12	0x23	0x34	0x45
	DECF	0x51,F		3	2	1	0
	BNZ	L1		Yes	Yes	Yes	No

POSTINC0 Highest Byte (0x113) à 0x45 0x34 0x23 0x12 Lowest Byte (0x110)

POSTINC2 Highest Byte (0x123) à 0x45 0x34 0x23 0x12 Lowest Byte (0x120)

The same program could be used to perform 8-Bit (1-Byte) up to 128-Bit (16-Bytes) data transfer just by changing the loop count value (Instruction 3) from 1 to 0x10 (16d). This

can be extended even further to over 128 bits if the FSR pointers are separated further apart beyond 16 locations.

Program 5.10 (a) Division by a factor of 2 using Rotate Instruction

The program performs division by using Rotate RRCF instruction using direct addressing mode. The 8-Bit data (0x42) from file register 0x30 is read and divided by 2 using RRCF. The result 0x21 is stored in file register 0x31.

MOVWF	0x30,W,A	; Move the value present in file register 0x30 ; to WREG, WREG = [0x30], WREG = 0x64
BCF	STATUS, 0	; Clear 0 th Bit – (Carry Flag) in Status Reg.
RRCF	0x30,W,A	; Rotate Right through Carry, store result in ; WREG, WREG = 0x32, CF = 0
MOVWF	0x31	; Store Divide by 2 result into file reg 0x31

Result

$$\begin{array}{rcl}
 & 0x42 & \beta \text{File Register 0x30} \\
 . & 0x02 & \text{RRCF} \\
 \hline
 = & 0x21 & \beta \text{File Register 0x31}
 \end{array}$$

Program 5.10 (b) Division by a factor of 16 using Rotate Instruction.

The program performs division of a number by 16 using Rotate RRCF instruction using direct addressing mode. The 8-Bit data (0x42) from file register 0x30 is read and divided by 16 using RRCF in a loop. The result 0x04 is stored in file register 0x30.

	MOVLW	0x04	; Count = 0x04; To perform Division by 16
	MOVWF	0x10	; Move Count to File Reg 0x10
	MOVWF	0x30, W, A	; Move the value present in file register 0x30 ; to WREG, WREG = [0x30], WREG = 0x64
L1	BCF	STATUS, 0	; Clear 0 th Bit – (Carry Flag) in Status Reg.
	RRCF	0x30, F, A	; Rotate Right through Carry, store result in ; File-Reg (0x30), File Reg value changes, ; divides by 2 every iteration ; Final result will be 0x04 in file register 0x30
	DECF	0x10, 1, 0	; Decrement File Register 0x10 content by 1, ; Store value in file reg.
	BNZ	L1	; Branch to L1 if zero flag is not set. Go to End ; if Zero. Loops 4 times to divide by 16.

Result

$$\begin{array}{rcl}
 & 0x64 & \beta \text{File Register 0x30} \\
 . & 0x16 & \text{RRCF x 4 times} \\
 \hline
 = & 0x04 & \beta \text{File Register 0x30}
 \end{array}$$

The same program could be used to perform division of 8-bit number by multiple factors of 2. i.e 2, 4, 8, 16 up to 256 by changing the loop count value in instruction 1 to a value between 1 and 8.

Similarly, multiplication of 8-bit number by multiple factors of 2. i.e 2, 4, 8, 16 up to 256 could be performed by replacing RRCF with RLCF instruction.

Program 5.11 Multiplication of 8-Bit numbers without MUL instruction.

The program performs multiplication of two 8-Bit numbers using direct addressing mode. It uses repeated addition technique instead of a MUL instruction to perform multiplication. Such logic is required in devices that do not have a MUL instruction like PIC16F877A. The 8-Bit data (0x81) and (0x82) are read from file registers 0x10 & 0x20 and repeated addition is performed through a loop. The result 0x41 0x82 is stored in file register 0x31 and 0x30 respectively.

	MOVF	0x20, 0	; Multiplier - No 1
	MOVWF	0x21	; Replicated for looping
	MOVF	0x10, 0	; Multiplicand - No 2
L2	ADDWF	0x30, 1, 0	; Low 8-Bit Result, store in File register 0x30
	BNC	L1	; Check if Carry, If value greater than 8-Bit
	INCF	0x31, 1, 0	; High 8-Bit Result, Increment if carry, store in
			; File register 0x31
L1	DECF	0x21, 1, 0	; Decrement M, store in File register 0x21
	BNZ	L2	; Loop till repeated addition based on
			; multiplier

Result

		<u>0x81</u>	βFile Register 0x10
		<u>0x82</u>	βFile Register 0x20
File Register 0x31à	0x41	0x82	βFile Register 0x30

Program 5.12 Division of 8-Bit/8-Bit numbers using repetitive subtraction

The program performs division of two 8-Bit numbers using direct addressing mode. It uses repeated subtraction technique, as no DIV instruction is available in PIC18F458 and PIC16F877A. The 8-Bit data (0x15) and (0x02) are read from file registers 0x10 & 0x20 and repeated subtraction is performed through a loop. The quotient 0x0A remainder 0x01 is stored in file register 0x30 and 0x31 respectively.

	MOVF	0x10,0	; No 1 8-Bit Dividend
	MOVWF	0x30	; Replicated for Subtraction/looping
L2	MOVF	0x20,0	; No 2 8-Bit Divisor

	CPFSGT	0x30,0	; If Greater perform Division
	BRA	L1	
	SUBWF	0x30,1,0	; Contains Remainder
	INCF	0x31,1,0	; Contains Quotient
	BRA	L2	
L1	NOP		

Result

		<u>0x15</u>	βFile Register 0x10
		<u>0x02</u>	βFile Register 0x20
File Register 0x31à (Remainder)	0x01	0x0A (Quotient)	βFile Register 0x30

Program 5.13(a) Multiplication of 16-Bit x 8-bit number to produce a 24-Bit result.

The program performs multiplication of a 16-Bit number with an 8-Bit number using direct addressing mode. It is an extension of program 5.11. The 16-Bit data (0x70 0x80) are read from file registers 0x11 & 0x10 and the 8-Bit data from 0x20. The result 0x3F 0x48 0x00 is stored in file register 0x32, 0x31 and 0x30 respectively.

; 16X8-Bit Multiplication with result of 24-Bit using repeated addition			
	MOVF	0x20, 0	; No 1 Low 8-Bit Multiplier
	MOVWF	0x40	; Replicated for looping
L2	MOVF	0x10, 0	; Low 8-Bit Multiplicand
	ADDWF	0x30, 1, 0	; Low (Bits 0 to Bit7) 8-Bit result
	MOVF	0x11, 0	; High 8-Bit Multiplicand
	ADDWFC	0x31, 1, 0	; Mid (Bits 8 to Bit15) 8-Bit result
	BNC	L1	
	INCF	0x32, 1, 0	; High (Bits 16 to Bit 23) 8-Bit result
L1	DECF	0x40, 1, 0	
	BNZ	L2	

Result

File Register 0x11à	0x70	0x80	βFile Register 0x10
		<u>0x90</u>	βFile Register 0x20
File Register 0x32à	0x3F	0x48	βFile Register 0x30
	File 0x31		
	Register		

Program 5.13(b) Division of 16-Bit/8-Bit numbers using repetitive subtraction

The program performs division of a 16-Bit number by an 8-Bit number using direct addressing mode. It is an extension of program 5.12.

The 16-Bit data (0xF0 0x01) are read from file registers 0x11 & 0x10 and the 8-Bit data from 0x20. The quotient 0x01 0x91 is stored in file register 0x33 and 0x32 while the 8-Bit remainder 0x58 is stored in 0x30.

; 16-Bit/8-Bit Division using repeated subtraction

```

MOVWF 0x10,0,0 ; No 1 Low 16-Bit Dividend
MOVWF 0x30,0 ; Replicated for Subtraction/looping
MOVWF 0x11,0,0 ; No 1 High 16-Bit Dividend
MOVWF 0x31 ; Replicated for Subtraction/looping
L2 MOVF 0x20,0,0 ; No 2 0x20 contains 8-Bit Divisor, WREG = [0x20]
    CPFSLT 0x30,0 ; If Greater or Equal to divisor perform Division
                ; i.e if not less than
    BRA L1
    BRA LN ; if divisor greater than low 8-bit dividend
                ; Branch to check higher 8-bit dividend
L1 MOVF 0x20,0,0 ; No 2 0x20 contains 8-Bit Divisor, WREG = [0x20]
    SUBWF 0x30,1,0 ; Repeated Subtraction, 0x30 Contains Remainder
    INCF 0x32,1,0 ; 0x32 Contains Lower 8-Bit Quotient
    BNC L51
    INCF 0x33,1,0 ; 0x33 contains Higher 8-Bit Quotient
L51 NOP
BRA L2
LN MOVLW 0x00 ; WREG = 0x00
    CPFSGT 0x31,0 ; If Higher 8-Bit Dividend greater than 0x00 perform
                ; Division
    BRA L3 ; End if not greater
    DECF 0x31,1,0 ; Decrement Higher 8-Bit Dividend by 1
    BRA L1
L3 NOP

```

Result

Sample Data 1

File Register 0x11à	0xF0	0x01	ßFile Register 0x10
	0x99		ßFile Register 0x20
File Register 0x33à	0x01	0x91	ßFile Reg 0x30 (Remainder)
High 8-Bit Quotient	File Reg		
	0x32, Low 8-Bit		
	Quotient		

Sample Data 2

File Register 0x11à	0x00	0x15	ßFile Register 0x10
	0x02		ßFile Register 0x20
File Register 0x33à	0x00	0x0A	ßFile Reg 0x30 (Remainder)

High 8-Bit Quotient

File Reg
0x32, Low 8-Bit
Quotient

Program 5.14 Packed BCD to Unpacked & then to ASCII

The program performs conversion of a packed BCD number to unpacked numbers and then to ASCII numbers.

This conversion is required when certain devices like LCD which takes data in ASCII form while the microprocessor deals in hexadecimal numbers, and the users require the data to be in the BCD form. The code reads the packed BCD number (given as 0x45 to PIC18) in file register 0x10 and converts to unpacked numbers (0x05, 0x04) and stores in 0x11 and 0x12. These are further converted to ASCII numbers (0x35, 0x34) and stored in 0x21 and 0x22.

A packed BCD number is a group of BCD numbers, i.e (0-9). Ex: A 2-digit packed BCD value ranges from 00 to 99. Unpacked number is a number in the range of 0-9 and represents between 0x00 to 0x09. ASCII is American Standard Code for Information interchange. Each character is assigned an ASCII number. The numbers 0-9 are ASCII coded as 0x30 to 0x39. The alphabets a – z is coded as 0x61 – 0x7A and A – Z are coded as 0x41 – 0x5A. Similarly, other special characters are assigned to a specific code. The ASCII code table is open-source data and readily available on the web.

MOVF	0x10,W,0	; WREG = [0x10] = 0x45, Packed BCD number
ANDLW	0x0F	; WREG = 0x05
MOVWF	0x11,0	; [0x11] = 0x05 Unpacked lower digit
MOVWF	0x21,0	; [0x21] = 0x05 Unpacked lower digit for ASCII ; conversion
MOVF	0x10,W,0	; WREG = [0x10] = 0x45, Packed BCD number
ANDLW	0xF0	; WREG = 0x40
MOVWF	0x12,0	; [0x12] = 0x40
SWAPF	0x12, 1, 0	; [0x12] = 0x04 Unpacked higher digit
MOVFF	0x12, 0x22	; [0x22] = 0x04 Unpacked higher digit for ASCII ; conversion
MOVLW	0x30	; WREG = 0x30
ADDWF	0x21,1,0	; [0x21] = 0x35 ASCII Lower digit
ADDWF	0x22,1,0	; [0x22] = 0x34 ASCII higher digit

Result

Packed BCD number	à 0x45	File Register 0x10
Unpacked lower digit	à 0x05	File Register 0x11
Unpacked higher digit	à 0x04	File Register 0x12

ASCII lower digit	à 0x35	File Register 0x21
ASCII higher digit	à 0x34	File Register 0x22

Program 5.15 ASCII to Unpacked BCD & then to Packed

The program performs conversion of group ASCII numbers to unpacked numbers and then to a packed BCD number.

The code reads the ASCII numbers (0x35, 0x34) in file register 0x11, 0x10; converts to unpacked numbers (0x05, 0x04) and stores in 0x22 and 0x21. These are further converted to packed BCD number (0x54) and stored in 0x30.

```

MOVWF 0x10,W,0 ; WREG = [0x10] = 0x34, Lower ASCII digit
ANDLW 0x0F      ; WREG = 0x04
MOVWF 0x21,0    ; [0x21] = 0x04, unpacked lower digit
MOVWF 0x11,W,0 ; WREG = [0x11] = 0x35, higher ASCII digit
ANDLW 0x0F      ; WREG = 0x05
MOVWF 0x22,0    ; [0x22] = 0x05, unpacked higher digit
MOVWF 0x30,0    ; [0x30] = 0x05, Unpacked higher digit for packing
SWAPF 0x30,1,0 ; [0x30] = 0x50 Unpacked higher digit for packing
MOVWF 0x11,W,0 ; WREG = 0x04
ADDWF 0x30,1,0 ; [0x30] = 0x54 packed higher digit

```

Result

ASCII lower digit	à 0x34	File Register 0x10
ASCII higher digit	à 0x35	File Register 0x11
Unpacked lower digit	à 0x04	File Register 0x21
Unpacked higher digit	à 0x05	File Register 0x22
Packed BCD number	à 0x54	File Register 0x30

Program 5.16 Number of 1's & 0's in a byte

The program counts the number of 1's and 0's present in a 8-Bit number. Such a count is helpful to determine the number of Input/Output - I/O's in ON state or the OFF state. Ex: Reading the switch status and determine the number of switches in ON or OFF position.

The code reads the 8-Bit value from a file register (0x20) that may have been loaded with a specific ports status value. The number of 1's is stored in file register 0x30 and number of 0's in 0x31.

```

BCF STATUS, 0 ; Clear Carry Flag
MOVLW 0x08    ; WREG = 0x08
MOVWF 0x10,0  ; [0x10] = WREG, Loop Count

```

	MOVWF	0x11,0	; [0x11] = WREG, Replication for Loop Count Variation
	MOVF	0x20,W,0	; WREG = [0x20], Indicate Starting Value
	MOVWF	0x21,0	; [0x21] = WREG, Replicated for Rotating
L3:	RLCF	0x21,F,0	; Rotating Value
	BNC	L1	; If Bit is 0 go to L1
	INCF	0x30,1,0	; No of 1's
	BRA	L2	
L1:	INCF	0x31,1,0	; No of 0's
L2:	DECF	0x11,F	
	BNZ	L3	

Result

Label	Op Code	Operand	Iteration							
			1	2	3	4	5	6	7	8
	BCF	STATUS, 0								
	MOVLW	0x08								
	MOVWF	0x10,0								
	MOVWF	0x11,0								
	MOVF	0x20,W,0	WREG = [0x20] = 0x43							
	MOVWF	0x21,0	[0x20] = 0x43							
L3:	RLCF	0x21,F,0	0x 86	0x 0C	0x 19	0x 32	0x 64	0x C8	0x 90	0x 21
	BNC	L1	CY = 0	1	0	0	0	0	1	1
	INCF	0x30,1,0	0	1	1	1	1	1	2	3
	BRA	L2								
L1:	INCF	0x31,1,0	1	1	2	3	4	5	5	5
L2:	DECF	0x11,F	7	6	5	4	3	2	1	0
	BNZ	L3	Yes	Y	Y	Y	Y	Y	Y	N

Given Number à 0x43 File Register 0x20

Number of 1's à 0x03 File Register 0x30

Number of 0's à 0x05 File Register 0x31

Program 5.17 Bit Set/Reset: Byte Wise

The purpose of this program is to demonstrate bit manipulation. Flags operations are common cases of bit manipulations. The other main usage of bit manipulations is for setting and resetting of peripheral control bits or I/O's. This program mainly demonstrates performing of bit manipulation on a byte of data to mimic cases where bit manipulation instructions as such are not present i.e like 8086 which only have flag-based bit

manipulations. Also, such logic helps in changing bit value in a byte when data can be read or written only as a byte like memory devices etc.

Here N1, N2, N3 & N4 are considered four memory locations values (numbers) that are read or written as a byte. B0 to B7 are the bit positions in a byte. N1B1 indicates Bit 1 in number 1, N4B5 indicates bit 5 in number 4 and so on.

```
if (N1B1 ==0)    {N3B3=0;}
else             {N3B3=1;}
if(N2B2==1)     {N4B5=0;}
else            {N4B5=1;}
```

	MOVFF	0x30,0x20	; Move number 3 memory value in file register ; [0x30] to [0x20]
	MOVFF	0x31,0x21	; Move number 4 memory value in file register ; [0x31] to [0x21]
	MOVF	0x10,W	; Move number 1 memory value in file register ; [0x10] to WREG
	ANDLW	0x02	; Check Bit 1 by performing an AND with 0x2
	BNZ	L1	; If result is 0, Bit 1 in number 1 is 0 else it is 1. ; Go to L1 to set Bit 3 in number 1 if result is not 0
	MOVLW	0xF7	; Make Bit 3 in number 1 to 0 by making bit 3 of W- ; Reg as 0
	ANDWF	0x30,1	; Reset Bit 3 in number 1 to 0 by AND with WREG, ; store result in file reg 0x30
	BRA	L2	; Branch to L2
L1:	MOVLW	0x08	; Make Bit 3 in number 1 to 1 by making bit 3 of W- ; Reg as 1
	IORWF	0x30,1	; Set Bit 3 in number 1 to 1 by OR with WREG, store ; result in file reg 0x30
L2:	MOVF	0x11,W	; Move number 2 memory value in file register ; [0x10] to WREG
	ANDLW	0x04	; Check Bit 2 by performing an AND with 0x2
	BZ	L3	; If result is 0, Bit 2 in number 2 is 0 else it is 1. ; Go to L3 to set Bit 3 in number 4 if result is not 0
	MOVLW	0xDF	; Make Bit 5 in number 4 to 0 by making bit 5 of W- ; Reg as 0
	ANDWF	0x31,1	; Reset Bit 5 in number 4 to 0 by AND with WREG, ; store result in file reg 0x31
	BRA	L4	; Branch to L4
L3:	MOVLW	0x20	; Make Bit 5 in number 4 to 1 by making bit 3 of W- ; Reg as 0
	IORWF	0x31,1	; Set Bit 5 in number 4 to 1 by OR with WREG, ; store result in file reg 0x31

L4: NOP

Result

		Data case1		Data case2
		76543210		76543210
No1:	[0x10] = 05	0000 0101	02	0000 0010
No2:	[0x11] = 04	0000 0100	00	0000 0000
No3:	[0x30] = 0F	0000 1111	07	0000 0111
No4:	[0x31] = 20	0010 0000	00	0000 0000
Afterwards				
No3:	[0x30] = 07	0000 0111	0F	0000 1111
No4:	[0x31] = 00	0000 0000	20	0010 0000

Program 5.18 Bit Set/Reset: Bit Wise

The purpose of this program is to demonstrate bit manipulation using bit-wise instructions. This logic helps in changing bit values in a byte when data can be read or written only as a byte like memory devices etc.

Here N1, N2, N3 & N4 are considered four memory locations values (numbers) that are read or written as a byte. B0 to B7 are the bit positions in a byte. N1B1 indicates Bit 1 in number 1, N4B5 indicates bit 5 in number 4 and so on.

```

if (N1B1 ==0)    {N3B3=0;}
else              {N3B3=1;}
if(N2B2==1)      {N4B5=0;}
else              {N4B5=1;}

```

	MOVFF	0x30,0x20	; Move number 3 memory value in file register ; [0x30] to [0x20]
	MOVFF	0x31,0x21	; Move number 4 memory value in file register ; [0x31] to [0x21]
	MOVF	0x10,W	; Move number 1 memory value in file register ; [0x10] to WREG
	BTFSC	WREG,1	; Check Bit 1 in number 1. Test bit 1 skip if bit is clear
	BRA	L1	; Branch to L1 if bit 1 is set
	BCF	0x30,3	; Clear bit 3 in number 3
	BRA	L2	; Branch to L2
L1:	BSF	0x30,3	; Clear bit 3 in number 3

L2:	MOVF	0x11,W	; Move number 2 memory value in file register ; [0x11] to WREG
	BTFSS	WREG,2	; Check Bit 2. Test bit 2 skip if bit is set
	BRA	L3	; Branch to L3 if bit 1 is clear
	BCF	0x31,5	; Clear bit 5 in number 4
	BRA	L4	; Branch to L4
L3:	BSF	0x31,5	; Clear bit 5 in number 4
L4:	NOP		; No Operation

Result

	Value	Data case1 76543210 (Bit no)	Value	Data case2 76543210 (Bit no)
No1:	[0x10] = 05	0000 0101	02	0000 0010
No2:	[0x11] = 04	0000 0100	00	0000 0000
No3:	[0x30] = 0F	0000 1111	07	0000 0111
No4:	[0x31] = 20	0010 0000	00	0000 0000
Afterwards				
No3:	[0x30] = 07	0000 0111	0F	0000 1111
No4:	[0x31] = 00	0000 0000	20	0010 0000

Program 5.19 Minimum/Maximum of given set of 8-Bit numbers

The program finds the maximum of a given set of 8-bit numbers starting from file register 0x20.

	MOVLW	0x04	; WREG = 0x04; Count - Number of 8-bit numbers
	MOVWF	0x10,0	; [0x10] = 0x04
	MOVWF	0x11,0	; [0x11] = 0x04, For looping
	LFSR	0,0x20	; FSR0 = 0x20
	MOVF	POSTINC0,0	; WREG = [0x20], First Number
L1	CPFSLT	INDF0	; (f) ? (W), skip if (f) < (W), For Largest
			; if [Next location value] < First, skip going to L3,
	;CPFSGT	INDF0	; (f) ? (W), skip if (f) > (W); For Smallest
	BRA	L3	
	MOVWF	0x12,0	; [0x12] = initial number
	BRA	L4	
L3	MOVF	INDF0,0	WREG = [Next location value]
	MOVWF	0x12,0	if Next number > First, [0x12] = next number
L4	MOVF	POSTINC0,1	; Go to next address
	DECF	0x11,F	; Reduce loop count
	BNZ	L1	; If loop count not 0 loop (all numbers checked)

NOP

Result

[0x10] = 0x04 No of numbers [0x11] = 0x04 Loop Count

[0x20] = 0x05

[0x21] = 0x08

[0x22] = 0x09

[0x23] = 0x03

Final Output [0x12] = 0x09

Program 5.20 Sorting in Ascending/Descending Order of a given set of 8-Bit numbers

The program finds the ascending of a given set of 8-bit numbers starting from file register 0x20.

	MOVLW	0x04	; WREG = 0x04; Count - Number of 8-bit numbers
	MOVWF	0x10,0	; [0x10] = 0x04
	MOVWF	0x12,0	; [0x12] = 0x04, For looping
	DECF	0x12,1,0	; Decrement count. 3 comparisons for 4 no's
L5	MOVFF	0x12,0x11	; [0x12] = 0x11, For restoring loop value
	LFSR	0,0x20	; FSR0 = 0x20
	MOVF	POSTINC0,0	; WREG = [0x20], First Number
L1	CPFSLT	INDF0	; (f) ? (W), skip if (f) < (W), For Descending
			; if [Next location value] < First, skip going to L3,
	;CPFSGT	INDF0	; (f) ? (W), skip if (f) > (W); For Ascending
	BRA	L3	; Branch to L3 for exchange if next > first
	MOVWF	0x13,0	; [0x13] = WREG, Temp storage
	BRA	L4	
L3	MOVWF	0x13,0	; [0x13] = WREG, Temp storage
	MOVF	INDF0,0	; WREG = [Next location]
	MOVFF	0x13,INDF0	; [Next location] = [0x13]
	MOVF	POSTDEC0,1	; Decrement location
	MOVWF	POSTINC0,0	; [Old location] = WREG,
			; [location back to normal]
	MOVF	INDF0,0	; Load WREG with [Next location] current
L4:	MOVF	POSTINC0,1	; Increment location
	DECF	0x11,F	; Decrement inner loop comparison's
	BNZ	L1	; If all comparison not done loop
	DECF	0x12,F	; Decrement outer loop comparison's
	BNZ	L5	; If all comparison not done loop
	NOP		

Result

[0x10] = 0x04 No of numbers [0x12] = 0x04
Later [0x12] = 0x03 Loop Count

Before

[0x20] = 0x03
[0x21] = 0x05
[0x22] = 0x08
[0x23] = 0x09

After Descending

0x09
0x08
0x05
0x03

Program 5.21 Branch Backwards Coding Example: Finding Offsets/Relative Value

The purpose of this program is to demonstrate the calculation of Relative/offset value during a backward jump. This logic helps in understanding hex codes, bytes used for instructions, change in PC value and calculation of relative value etc that may be used to decide whether to use a branch (11-Bit jump) or goto (20-Bit jump) instruction. i.e 2K jump for branch and 1MB or anywhere jump for goto.

```
1      MOVLW 0x02      ; WREG = 0x02
2      MOVWF 0x00,0    ; [0x00] = 0x02
3      L1      MOVLW 0x04      ; WREG = 0x04
4      MOVWF 0x01,0    ; [0x00] = 0x01
5      ADDWF 00H,0,0    ; WREG = [0x00] + WREG (0x01);
                        ; i.e WREG = 0x02 + 0x04 = 0x06
6      BNC     L1        ; No Carry, So Go to L1
7      MOVLW 0x07      ; WREG = 0x07
8      MOVWF 0x03,0    ; [0x03] = WREG
```

			Line	Address	Opcode	Label	DisAssey
2	ORG	0x00					
3	GOTO	START	1	0000	EF02		GOTO 0x4
4	START		2	0002	F000		NOP
5	MOVLW	0x02	3	0004	0E02	START	MOVLW 0x2
6	MOVWF	0x00,0	4	0006	6E00		MOVWF 0x0, A
7	MOVLW	0x04	5	0008	0E04		MOVLW 0x4
8	MOVWF	0x01,0	6	000A	6E01		MOVWF 0x1, A
9	ADDWF	00H,0,0	7	000C	2400		ADDWF 0x0, W
10	BNC	L1	8	000E	E3FF	L1	BNC L1
11	MOVLW	0x07	9	0010	0E07		MOVLW 0x7
12	MOVWF	0x03,0	10	0012	6E03		MOVWF 0x3, A
			11	0014	0000		NOP
			12	0016	FFFF		NOP

Case (a) When L1 label is at 6 instruction BNC L1; the offset value is -1 or FF. When BNC L1 is getting executed, PC actually fetches MOVLW 0x07 instruction 7. To go back to instruction 6, the offset now becomes -1 or two's complement value FF with E3 being the hex code for the BNC instruction.

6	MOVWF 0x00,0	4	0006	6E00		MOVWF 0x0, A
7	MOVLW 0x04	5	0008	0E04		MOVLW 0x4
8	MOVWF 0x01,0	6	000A	6E01		MOVWF 0x1, A
	L1 ADDWF 00H, 0, 0	7	000C	2400	L1	ADDWF 0x0, W
	BNC L1	8	000E	E3FE		BNC L1
11	MOVLW 0x07	9	0010	0E07		MOVLW 0x7

Case (b) When L1 label is at 5 instruction ADDWF 00H, 0, 0; the offset value is -2 or FE. When BNC L1 is getting executed, PC fetches MOVLW 0x07 instruction 7. To go back to instruction 5, the offset now becomes -2 or two's complement value FE.

6	MOVWF 0x00,0	4	0006	6E00		MOVWF 0x0, A
7	MOVLW 0x04	5	0008	0E04		MOVLW 0x4
	L1 MOVWF 0x01,0	6	000A	6E01	L1	MOVWF 0x1, A
9	ADDWF 00H, 0, 0	7	000C	2400		ADDWF 0x0, W
	BNC L1	8	000E	E3FD		BNC L1
11	MOVLW 0x07	9	0010	0E07		MOVLW 0x7

Case (c) When L1 label is at 4 instruction MOVWF 0x01, 0; the offset value is -3 or FD. When BNC L1 is getting executed, PC actually fetches MOVLW 0x07 instruction 7. To go back to instruction 4, the offset now becomes -3 or two's complement value FD.

6	MOVWF 0x00,0	4	0006	6E00		MOVWF 0x0, ACCE
	L1 MOVLW 0x04	5	0008	0E04	L1	MOVLW 0x4
8	MOVWF 0x01,0	6	000A	6E01		MOVWF 0x1, ACCE
9	ADDWF 00H, 0, 0	7	000C	2400		ADDWF 0x0, W, A
	BNC L1	8	000E	E3FC		BNC L1
11	MOVLW 0x07	9	0010	0E07		MOVLW 0x7

Case (d) When L1 label is at 3 instruction MOVLW 0x04; the offset value is -4 or FC. When BNC L1 is getting executed, PC fetches MOVLW 0x07 instruction 7. To go back to instruction 3, the offset now becomes -4 or two's complement value FC.

Note: The instructions used between branch instruction BNC L1 and the label L1 have all been a 1 Word instructions. If a 2-word instruction like MOVFF or GOTO instructions were present, the offset increases accordingly. i.e. 1 offset value extra for each 2-word instruction need to be considered while calculating the offset.

Program 5.22 Branch Forward Coding Example: Finding Offsets/Relative Value

The purpose of this program is to demonstrate the calculation of Relative/offset value during a forward jump. This logic helps in understanding hex codes, bytes used for instructions, change in PC value and calculation of relative value etc that may be used to decide whether to use a branch (11-Bit jump) or goto (20-Bit jump) instruction. i.e 2K jump for branch and 1MB or anywhere jump for goto.

1	MOVLW	0x02	; WREG = 0x02
2	MOVWF	0x00,0	; [0x00] = 0x02
3	ADDLW	05H	; WREG = 0x05 + WREG (0x02); ; i.e WREG = 0x05 + 0x02 = 0x07
4	BNC	L1	; No Carry, So Go to L1

5	MOVLW	0x04	; WREG = 0x04
6	MOVWF	0x01,0	; [0x01] = 0x04
7	MOVLW	0x07	; WREG = 0x07
8	L1	MOVWF	0x03,0 ; [0x03] = WREG
9	NOP		

		Line	Address	Opcode	Label	DisAssy
2	ORG	0x00				
3	GOTO	START	1	0000	EF02	GOTO 0x4
4	START		2	0002	F000	NOP
5	MOVLW	0x02	3	0004	0E02	START MOVLW 0x2
6	MOVWF	0x00,0	4	0006	6E00	MOVWF 0x0, A
7	ADDLW	05H	5	0008	0F05	ADDLW 0x5
8	BNC	L1	6	000A	E301	BNC L1
9	MOVLW	0x04	7	000C	0E04	MOVLW 0x4
10	L1	MOVWF	8	000E	6E01	L1 MOVWF 0x1, A
11	MOVLW	0x07	9	0010	0E07	MOVLW 0x7
12	MOVWF	0x03,0	10	0012	6E03	MOVWF 0x3, A
			11	0014	0000	NOP

Case (a) When L1 label is at 5 instruction MOVLW 0x04; the offset value is 0. When BNC L1 is getting executed, PC actually fetches the next instruction at 5. So offset is 0.

If the L1 label is at 6 instruction MOVWF 0x01, since during execution of instruction 4, PC will point to instruction 5, the offset is now 0x01.

7	ADDLW	05H	5	0008	0F05	ADDLW 0x5
8	BNC	L1	6	000A	E302	BNC L1
9	MOVLW	0x04	7	000C	0E04	MOVLW 0x4
10	MOVWF	0x01,0	8	000E	6E01	MOVWF 0x1, A
11	L1	MOVLW	9	0010	0E07	L1 MOVLW 0x7
12	MOVWF	0x03,0	10	0012	6E03	MOVWF 0x3, A

Case (b) When L1 label is at 7 instruction MOVLW 0x07; the offset value is 2. When BNC L1 is getting executed, PC fetches MOVLW 0x04 instruction 5. To go to instruction 7, the offset now becomes 2.

7	ADDLW	05H	5	0008	0F05	ADDLW 0x5
8	BNC	L1	6	000A	E303	BNC L1
9	MOVLW	0x04	7	000C	0E04	MOVLW 0x4
10	MOVWF	0x01,0	8	000E	6E01	MOVWF 0x1, A
11	MOVLW	0x07	9	0010	0E07	MOVLW 0x7
12	L1	MOVWF	10	0012	6E03	L1 MOVWF 0x3, A
			11	0014	0000	NOP

Case (c) When L1 label is at 8 instruction MOVWF 0x03, 0; the offset value is 3. When BNC L1 is getting executed, PC fetches MOVLW 0x04 instruction 5. To go to instruction 8, the offset now becomes 3.

Note: The instructions used between branch instruction BNC L1 and the label L1 have all been a 1 Word instructions. If a 2-word instruction like MOVFF or GOTO instructions were present, the offset increases accordingly. i.e. 1 offset value extra for each 2-word instruction need to be considered while calculating the offset.

5.1.2 Bank Switching

As already discussed in chapter 4, PIC18 data memory is divided into different banks. Overall, a total of 16 banks and 1 default access bank is possible in PIC18 series. PIC18F458 has 7 (6 general purpose and one SFR) banks and 1 access bank.

Program 5.23(a) Multi Bank 8-Bit Addition

The purpose of this program is to demonstrate how data can be accessed from different banks.

A simple 8-bit addition program discussed earlier is used for this purpose considering data in different banks. Here each 8-bit data required to perform addition are in bank 0 and bank 1 with results in bank 2. i.e file register 0x10, 0x110 and 0x210 respectively.

```

MOVLB    0x0      ; Selecting Bank 0, BSR = 0x00
MOVWF    0x10,0,0 ; WREG = [0x10], accessing No 1 from bank 0
MOVLB    0x1      ; Selecting Bank 1, BSR = 0x01
ADDWF    0x10,0,1 ; WREG = [0x110] + WREG, accessing No 2 from bank 1
              ; and perform addition, store result in in WREG
MOVLB    0x2      ; Selecting Bank 2, BSR = 0x02
MOVWF    0x10,1   ; [0x210] = WREG, store result into bank 2

```

Result

No1:	Bank 0	[0x010]	No1:	Bank 0	[0x010] = 0x23
No2:	Bank 1	[0x110]	No2:	Bank 1	[0x110] = 0x34
Result:	Bank 2	[0x210]	Result:	Bank 2	[0x210] = 0x57

Program 5.23(b) Multi Bank 16-Bit Addition

In this program, each 16-bit data required to perform addition is in bank 0 and bank 1 with results in bank 2. i.e. file register 0x11, 0x10 containing 16-bit number1, 0x111, 0x110 containing 16-bit number 2 and 0x212, 0x211 containing the 16-bit result respectively.

```

MOVLB    0x0      ; Selecting Bank 0, BSR = 0x00
MOVWF    0x10,0   ; WREG = [0x10], accessing No 1 low 8-bit from
              ; bank 0
MOVLB    0x1      ; Selecting Bank 1, BSR = 0x01
ADDWF    0x10,0,1 ; WREG = [0x110] + WREG, accessing No 2 low 8-bit
              ; from bank 1 and perform addition, store result in in
              ; WREG
MOVLB    0x2      ; Selecting Bank 2, BSR = 0x02
MOVWF    0x10,1   ; [0x210] = WREG, store low 8-bit result into bank 2
MOVLB    0x0      ; Selecting Bank 0, BSR = 0x00 ;
MOVWF    0x11,0,0 ; WREG = [0x11], accessing No 1 high 8-bit
              ; from bank 0
MOVLB    0x1      ; Selecting Bank 1, BSR = 0x01 ;
ADDWF    0x11,0,1 ; WREG = [0x111] + WREG, accessing No 2 high

```

		; 8- bit from bank 1
MOVLB	0x2	; Selecting Bank 2, BSR = 0x02
MOVWF	0x11,1	; [0x210] = WREG, store result into bank 2

Result

No1: Bank 0 [0x010] = 0x23 [0x011] = 0x56

No2: Bank 1 [0x110] = 0x34 [0x111] = 0x78

Result: Bank 2 [0x210] = 0x57 [0x211] = 0xCE

The same could be achieved without changing the bank using the multi-byte addition program demonstrated earlier in Program 5.8 through 12-Bit file register indirect memory access by pointing FSR0 to 0x010, FSR1 to 0x110 and FSR2 to 0x210 with a count of 2.

Program 5.24 Accessing SFR's across banks in direct addressing mode.

The purpose of this program is to demonstrate accessing SFR's across banks. Here the PORTB SFR is used for this purpose whose address is 0xF81 and the address of WREG is 0xFE8.

MOVLB	0X02	; Selecting Bank 2, BSR = 0x02
MOVLW	B'00000000'	; WREG = 0x00
MOVWF	TRISB,0	; TRISB = 0x00, Make PORTB as output.
MOVLW	0X05	; WREG = 0x05
MOVWF	0x81,1	; Move WREG Value 0x05 to bank 2 file reg 0x81, ; i.e. file register 0x281
MOVWF	0X81,0	; Move WREG Value 0x05 to access bank file reg ; 0x81, or PORTB
MOVLW	0X09	; WREG = 0x09
MOVFF	0xFE8, 0XF81	; Move WREG Value 0x09 to SFR addressed 0xF81, ; i.e. PORTB, using 12-Bt address.
MOVLW	0X06	; WREG = 0x06
MOVWF	PORTB,0	; Move WREG Value 0x06 to SFR 0xF81, ; i.e. PORTB, using SFR name
MOVLW	0X07	; WREG = 0x07
MOVWF	PORTB,1	; Move WREG Value 0x06 to bank 2 file reg, 0x81, ; i.e. file register 0x281, not PORTB
NOP		; No Operation

5.1.3 Macros & Subroutines

An application program is generally divided into group of subprograms and organized hierarchically. When a group of instructions need to be used repeatedly, they are grouped together. This set of instructions may be written as macros or subroutines.

Subroutine:

When an instruction group has a greater number of instructions and needs to be repeated, they are written as a subroutine. A subroutine is similar to functions in high level languages like C. In assembly language they are written as

- Subroutines using assembly language instructions or
- Subroutines as procedures using assembler directives (Depends on the assembler used, may or may not be supported)

Subroutines created using assembly language instructions are invoked using a “CALL” instruction in the main program, has an identification label name to indicate the start of a subroutine and require a “RETURN” instruction at the end of subroutine to indicate the end of subroutine. Subroutines are invoked by the execution of instructions in CPU. Subroutines are created using assembler directive like “PROCEDURE” to indicate the start of a procedure and “ENDP” to indicate the end of a procedure. The names of the assembler directives may vary from one assembler to other.

Instructions written in subroutine are not replicated and are also not stored in program memory each time they are required. They are only stored in program memory once at a certain locality. Whenever a subroutine is called/invoked from anywhere in the main program, this locality in program memory is accessed and executed by the CPU.

Since instructions are stored in the program memory only once, the number of memory bytes required is reduced. As an example, if a subroutine is having 10 instructions and is invoked 20 times, the amount of memory required for 10 instructions, if assumed as 1 word per instruction, requires only 10 words (20 bytes). The same if not written as a subroutine requires 200 words (400 bytes) since the same instructions need to be stored in program memory 20 times. This is the main advantage of using subroutines. Also, the program written in the editor by the user will most certainly be the same as the one loaded into the program memory of the microcontroller/microprocessor (unless it's optimized by the assembler).

The main disadvantage of subroutines is that every call instruction creates a jump to the location of subroutine and another jump to come back to main program when a return is encountered at the end of subroutine. So, every execution of a subroutine causes two jumps to and from subroutine from the main program. This causes an overhead, additional execution time/steps along with requirement of context saving. Context saving is the process to temporarily store the content of PC on to stack during a call and restoring it back to PC from stack during a return instruction. It also requires flushing of the pipeline (discard the already fetched instruction). Since PIC18 has a 2-stage pipeline it discards the one already fetched instruction. Program 5.25(a & b) is used to demonstrate the use of subroutines. An

example demonstration of calculator application with and without subroutine is presented in program 5.26 and 5.27.

Passing values to subroutines.

There are different ways of passing values to subroutines. Legacy devices (8086 or 8051 etc) make use of the following.

- Passing parameters through Registers
- Passing parameters through Data Memory, accessed directly through address.
- Passing parameters through Data Memory, accessed indirectly/indexed/relative mode through pointers.
- Passing parameters through Stack Memory

PIC18458 employs similar methods as above with a few differences.

- Passing parameters through Registers
 - PIC18F458 does not have general purpose registers as such except for the WREG which is the default register that is also address mapped in SFR section. Some physical readable and writable SFRs (Ex: Timer register - T0CON etc) may be used as general purpose registers if the respective special functionality is not being used. This is demonstrated in program 5.25(c).
- Passing parameters through Data Memory, accessed directly through address.
 - As discussed, earlier PIC18F458 makes use of file registers that are address mapped in data memory. So PIC18F458 by default operates data memory using direct addressing method. File registers 0x000 to 0x5FF and SFR's 0xF00 to 0xFFF. Data can be accessed through specific address, bank based 8-bit address or non-bank based 12-bit address. This is demonstrated in program 5.25(d)
- Passing parameters through Data Memory, accessed indirectly/indexed/relative mode through pointers.
 - PIC18F458 makes use of file select registers FSR0, FSR1 & FSR2 which act as pointers and uses non-physical registers INDF, POSTINC, POSTDEC, PREINC or PLUSW etc to access the file register address indirectly or in indexed, relative mode. This is demonstrated in program 5.25(e). Additionally, program 5.25 (f & g) demonstrate the use of assembler directives Variable and SET directives which accesses based on direct addressing. The assembler directive just replaces the variable name with the equivalent file register address.
- Passing parameters through Stack Memory

- PIC18 does not have a dedicated stack for temporary data. The one level fast stack is only for WREG, STATUS & BSR. It only has 31-level stack for storing PC containing program memory locations. This could be used to pass data to subroutines through TOS – Top of Stack, by using PUSH, modify TOS and POP etc. This method has limited use, store max of 30 values, it is not straight forward, care and additional instruction steps are need & ineffective as its not designed for data passing. The Fast Save stack-based example is presented in program 5.28

Program 5.25(a) A simple example of subroutine that performs 8-Bit Addition, (without passing of parameters)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.1(b).

MY_ADD	MOVF	0x10, 0, 0	; Get an 8-Bit value from file register 0x10
	ADDWFC	0x20, 0, 0	; Get an 8-Bit value from file register 0x20, add ; to value WREG and store result back into W
	MOVWF	0x30, 0	; Move the result in WREG to file register 0x30
	RETURN		; Return to main from subroutine
START	; Main		; Call addition subroutine
	CALL	MY_ADD	

Result

0x34 ßFile Register 0x10
0x23 ßFile Register 0x20
0x57 ßFile Register 0x30

Here no parameters are passed to or from the subroutine.

Program 5.25(b) A simple example of subroutine that performs 16-Bit Addition, (without passing of parameters)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.2(b).

MY_SR			
	MOVF	POSTINC0, 0, 0	; Get an 8-Bit value from file register pointed ; by FSR0, store in WREG
	ADDWFC	POSTINC1, 0, 0	; Get an 8-Bit value from file register pointed ; by FSR1, add to value WREG and store ; result back into WREG
	MOVWF	POSTINC2, 0	; Move the result in WREG to file register ; pointed by FSR2

	RETURN		; Return to main from subroutine
START	LFSR	0, 0x110	; Set the pointer 1, FSR0 = 0x110
	LFSR	1, 0x120	; Set the pointer 2, FSR1 = 0x120
	LFSR	2, 0x130	; Set the pointer 3, FSR2 = 0x130
	CALL	MY_SR	; Call MY_SR Subroutine
	CALL	MY_SR	; Call MY_SR Subroutine
	NOP		; No Operation

			Address	HEX Code	Label	Instructions
	ORG	0X30	0030	50EE		MOVWF POSTINC0, W, ACCESS
MY_SR			0032	20E6		ADDWFC POSTINC1, W, ACCESS
	MOVWF	POSTINC0, 0, 0	0034	6EDE		MOVWF POSTINC2, ACCESS
	ADDWFC	POSTINC1, 0, 0	0036	0012		RETURN 0
	MOVWF	POSTINC2, 0	0038	EE01		LFSR 0, 0x110
	RETURN		003A	F010		NOP
			003C	EE11		LFSR 1, 0x120
			003E	F020		NOP
START	LFSR	0, 0x110	0040	EE21		LFSR 2, 0x130
	LFSR	1, 0x120	0042	F030		NOP
	LFSR	2, 0x130	0044	EC18		CALL 0x30, 0
	CALL	MY_SR	0046	F000		NOP
	CALL	MY_SR	0048	EC18		CALL 0x30, 0
	NOP		004A	F000		NOP

Result

File Register 0x111 à 0x56 0x34 ßFile Register 0x110

File Register 0x121 à 0x45 0x23 ßFile Register 0x120

File Register 0x131 à 0x9B 0x57 ßFile Register 0x130

As can be seen from the figure the subroutine is stored only once in the memory from 0x30 to 0x37 but invoked twice in 0x44 and 0x48.

Program 5.25(c) A simple example of subroutine that performs 8-Bit Addition, (passing of parameters using registers)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.1(b).

MY_ADD			
	ADDWF	TOCON, 0, 0	; Get an 8-Bit value from file register TOCON, add ; to value WREG and store result back into W ; WREG = TOCON + WREG
	MOVWF	0x30, 0	; Move the result in WREG to file register 0x30
	RETURN		; Return to main from subroutine
START	; Main		; Call addition subroutine
	MOVF	0x10, 0, 0	; WREG = [0x10], Get No 1
	MOVWF	TOCON, 0	; Move the 8-Bit value to TOCON

MOVF	0x20, 0, 0	; WREG = [0x20], Get No 2
CALL	MY_ADD	

Result

0x34 T0CON ßFile Register 0x10

0x23 WREG ßFile Register 0x20

0x57 ßFile Register 0x30

Here no parameters are passed to or from the subroutine.

Program 5.25(d) A simple example of subroutine that performs 8-Bit Addition, (with passing of parameters in file register, accessed using direct addressing mode in subroutine)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.1(b). Here the file registers 0x10, 0x20, 0x30 are loaded with values using immediate addressing mode instructions in the main program and are accessed in subroutine using direct addressing mode.

MY_ADD	MOVF	0x10, 0, 0	; Get an 8-Bit value from file register 0x10
	ADDWF	0x20, 0, 0	; Get an 8-Bit value from file register 0x20, add ; to value WREG and store result back into W
	MOVWF	0x30, 0	; Move the result in WREG to file register 0x30
	RETURN		; Return to main from subroutine
START	; Main		
	MOVLW	0x34	; WREG = 0x34
	MOVWF	0x10, 0	; [0x10] = WREG = 0x34
	MOVLW	0x23	; WREG = 0x23
	MOVWF	0x20, 0	; [0x20] = WREG = 0x23
	CALL	MY_ADD	; Call addition subroutine

Result

0x34 ßFile Register 0x10

0x23 ßFile Register 0x20

0x57 ßFile Register 0x30

Program 5.25(e) A simple example of subroutine that performs 8-Bit Addition, (with passing of parameters in file register, accessed using pointers - indirect/index addressing mode in subroutine)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.1(c). Here data is stored in file registers in main program using immediate addressing mode instructions and is later utilized through indirect addressing mode in subroutine. The pointer 1, 2 & 3 values of 0x110, 0x120, 0x130 are passed through physical registers FSR0, FSR1 & FSR2 and the respective addresses

are accessed through non-physical registers POSTINC0, POSTINC1 AND POSTINC2 in the subroutine in indirect/indexed addressing mode.

MY_ADD	MOVF	POSTINC0, 0, 0	; Get an 8-Bit value from file register pointed ; by FSR0, store in WREG
	ADDWFC	POSTINC1, 0, 0	; Get an 8-Bit value from file register pointed ; by FSR1, add to value WREG and store ; result back into WREG
	MOVWF	POSTINC2, 0	; Move the result in WREG to file register ; pointed by FSR2
	RETURN		; Return to main from subroutine
START	MOVLB	0X01	
	MOVLW	0x34	; WREG = 0x34
	MOVWF	0x10, 1	; [0x10] = WREG = 0x34
	MOVLW	0x23	; WREG = 0x23
	MOVWF	0x20, 1	; [0x20] = WREG = 0x23
	LFSR	0, 0x110	; Load FSR0 with 0x110
	LFSR	1, 0x120	; Load FSR1 with 0x120
	LFSR	2, 0x130	; Load FSR2 with 0x130
	BCF	STATUS, 0	; Clear Carry flag in STATUS register
	CALL	MY_ADD	; Call addition subroutine

Result

0x34 ßFile Register 0x110
0x23 ßFile Register 0x120
0x57 ßFile Register 0x130

Program 5.25(f) A simple example of subroutine that performs 8-Bit Addition, (with passing of parameters using variables – direct addressing)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.1(b). Here the variables VAR1, VAR2 and VAR3 created using Variable directive are used in place of file registers. The values passed into the subroutine are the values that are assigned before the subroutine definition and not before the subroutine call. To demonstrate this VAR1, VAR2 & VAR3 are assigned 3 different values in different places in the program file. 0x10, 0x20 and 0x30 are assigned before the definition of MY_ADD1 subroutine in line 2,3 and 4. 0x11, 0x21 and 0x31 are assigned after the definition of MY_ADD1 subroutine in line 12,13 and 14 before the MY_ADD1 subroutine is called. 0x12, 0x22 and 0x32 are assigned before the definition of MY_ADD2 subroutine in line 19,20 and 21 after the MY_ADD2 subroutine is called but assigned before the definition of MY_ADD2 subroutine. The values assigned to the variables

before the definition of the subroutine are the values considered and not the values assigned before the subroutine call.

1		VARIABLE	VAR1, VAR2, VAR3	; Declaring Variable VAR1, VAR2 & ; VAR3 for 8-Bit No 1, 2 & Result
2		VAR1 =	0x10	; VAR1 = 0x10, Address of No1
3		VAR2 =	0x20	; VAR2 = 0x20, Address of No 2
4		VAR3 =	0x30	; VAR3 = 0x30, Address of Result
5	MY_ADD1			
6		MOVF	VAR1,0,0	; WREG = [VAR1], VAR1 = 0x10, ; WREG = 0x01
7		ADDWF	VAR2,0,0	; WREG = [VAR2] + WREG, ; VAR2 = 0x20 ; WREG = 0x05 + 0x01 = 0x06
8		MOVWF	VAR3,0	; [VAR3] = WREG, VAR3 = 0x30, ; [0x30] = 0x06
9		NOP		; No Operation
10		RETURN		; Return
11	START			
12		VAR1 =	0x11	; VAR1 = 0x11, Address of No1, ; not considered
13		VAR2 =	0x21	; VAR2 = 0x21, Address of No 2 ; not considered
14		VAR3 =	0x31	; VAR3 = 0x31, Address of Result ; not considered
15		CALL	MY_ADD1	; Call MY_ADD1 subroutine
16		CALL	MY_ADD2	; Call MY_ADD2 subroutine
17		NOP		; No Operation
18	HERE	BRA	HERE	; Keep looping here
19		VAR1 =	0x12	; VAR1 = 0x12, Address of No1
20		VAR2 =	0x22	; VAR2 = 0x22, Address of No 2
21		VAR3 =	0x32	; VAR3 = 0x32, Address of Result
22	MY_ADD2			
23		MOVF	VAR1,0,0	; WREG = [VAR1], VAR1 = 0x12, ; WREG = 0x03
24		ADDWF	VAR2, 0,0	; WREG = [VAR2] + WREG, VAR2 = 0x22 ; WREG = 0x07 + 0x03 = 0x0A
25		MOVWF	VAR3,0	; [VAR3] = WREG, VAR3 = 0x32, ; [0x32] = 0x0A
26		NOP		; No Operation
27		RETURN		; Return

File Registers x					
Address	00	01	02	03	04
000	00	00	00	00	00
010	01	02	03	04	00
020	05	06	07	08	00
030	06	22	0A	44	00
040	00	00	00	00	00

<u>Result</u>	Subroutine MY_ADD1		Subroutine MY_ADD2
0x01	βFile Register 0x10	0x03	βFile Register 0x12
<u>0x05</u>	<u>βFile Register 0x20</u>	<u>0x07</u>	<u>βFile Register 0x22</u>
0x06	βFile Register 0x30	0x0A	βFile Register 0x32

Program 5.25(g) A simple example of subroutine that performs 8-Bit Addition, (with passing of parameters using SET directive – direct addressing)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.1(b). Here the variables VAR1, VAR2 and VAR3 created using set directive are used in place of file registers. The values passed into the subroutine are the values that are assigned before the subroutine definition and not before the subroutine call. The only difference is SET directive is used instead of variable directive.

```

VAR1    SET    0x10    ; VAR1 = 0x10, Address of No1
VAR2    SET    0x20    ; VAR2 = 0x20, Address of No 2
VAR3    SET    0x30    ; VAR3 = 0x30, Address of Result
MY_ADD1                                ; Considers the above values not the values
                                      ; before subroutine call
      MOVF     VAR1,0,0    ; WREG = [VAR1], VAR1 = 0x10,
                          ; WREG = 0x01
      ADDWF    VAR2,0,0    ; WREG = [VAR2] + WREG,
                          ; VAR2 = 0x20
                          ; WREG = 0x05 + 0x01 = 0x06
      MOVWF    VAR3,0      ; [VAR3] = WREG, VAR3 = 0x30,
                          ; [0x30] = 0x06
      NOP      ; No Operation
      RETURN   ; Return

START
VAR1    SET    0x11    ; VAR1 = 0x11, Address of No1,
                      ; not considered
VAR2    SET    0x21    ; VAR2 = 0x21, Address of No 2
                      ; not considered
VAR3    SET    0x31    ; VAR3 = 0x31, Address of Result
                      ; not considered

```

```

CALL    MY_ADD1    ; Call MY_ADD1 subroutine
CALL    MY_ADD2    ; Call MY_ADD2 subroutine
NOP      ; No Operation
HERE    BRA        HERE    ; Keep looping here
VAR1    SET        0x12    ; VAR1 = 0x12, Address of No1
VAR2    SET        0x22    ; VAR2 = 0x22, Address of No 2
VAR3    SET        0x32    ; VAR3 = 0x32, Address of Result
MY_ADD2
MOVWF   VAR1,0,0    ; WREG = [VAR1], VAR1 = 0x12,
                   ; WREG = 0x03
ADDWFF  VAR2, 0,0    ; WREG = [VAR2] + WREG, VAR2 = 0x22
                   ; WREG = 0x07 + 0x03 = 0x0A
MOVWF   VAR3,0      ; [VAR3] = WREG, VAR3 = 0x32,
                   ; [0x32] = 0x0A
NOP      ; No Operation
RETURN   ; Return

```

File Registers x					
Address	00	01	02	03	04
000	00	00	00	00	00
010	01	02	03	04	00
020	05	06	07	08	00
030	06	22	0A	44	00
040	00	00	00	00	00

<u>Result</u>	Subroutine MY_ADD1	Subroutine MY_ADD2
0x01	βFile Register 0x10	0x03 βFile Register 0x12
<u>0x05</u>	<u>βFile Register 0x20</u>	<u>0x07</u> βFile Register 0x22
0x06	βFile Register 0x30	0x0A βFile Register 0x32

Program 5.26 Calculator without subroutines across banks

```

START    MOVF      00H,0,0    ; Get the calculator input choice.
                                ; If the choice is 1 perform addition, for a choice
                                ; 2 perform subtraction, for a choice 3 perform
                                ; multiplication.
                                CPFSEQ  01H,0    ; Compare to see if choice is 1 or not.
                                ; If not 1 go to check if choice is 2 or 3.
                                BRA      L1        ; Choice is not 1 so go to check if choice is 2 or 3.
                                MOVLB   0x1        ; Set bank 1 and perform addition.
                                MOVF    0x10,W      ; WREG = [0x110]
                                ADDWFF  0x20,W      ; WREG = [0x120] + WREG
                                MOVWF   0x30        ; [0x130] = WREG
                                BRA      LEND       ; Go to end after completion of addition.

```

L1	CPFSEQ	02H,0	; Compare to see if choice is 2 or not if its not 1. ; If not 2 go to check if choice is 3.
	BRA	L2	; Choice is not 2 so go to check if choice is 3.
	MOVLB	0x02	; Set bank 2 and perform subtraction.
	MOVF	0x40,W	; WREG = [0x240]
	SUBWF	0x50,W	; WREG = [0x250] + WREG
	MOVWF	0x60	; [0x260] = WREG
L2	BRA	LEND	; Go to end after completion of subtraction.
	CPFSEQ	03H,0	; Compare to see if choice is 3 or not if its not 1,2. ; If not 3 go to end, as its wrong choice.
	BRA	LEND	; Choice is not 3 also, so entered choice is wrong. ; Go to end.
	MOVLB	0x03	; Set bank 3 and perform multiplication.
	MOVF	0x70,W,A	; WREG = [0x370]
	MULWF	0x80	; WREG = [0x380] + WREG
LEND	MOVFF	0xFF3,0x090	; [0x390] = PRODL
	MOVFF	0xFF4,0x091	; [0x391] = PRODH
LEND	NOP		; No Operation.

Result

ADD		SUB	
0x02	βFile Register 0x110	0x02	βFile Register 0x240
0x03	βFile Register 0x120	0x03	βFile Register 0x250
0x05	βFile Register 0x130	0xFF	βFile Register 0x260
MUL			
0x02	βFile Register 0x370		
0x03	βFile Register 0x380		
0x06	βFile Register 0x390 PRODL		
0x00	βFile Register 0x391 PRODH		

Program 5.27 Calculator with subroutines across banks

MY_ADD	MOVF	0x10,W	; WREG = [0x110]
	ADDWF	0x20,W	; WREG = [0x120] + WREG
	MOVWF	0x30	; [0x130] = WREG
	RETURN		; Return back to main to branch to end
MY_SUB	MOVF	0x40,W	; WREG = [0x240]
	SUBWF	0x50,W	; WREG = [0x250] + WREG
	MOVWF	0x60	; [0x260] = WREG
	RETURN		; Return back to main to branch to end

MY_MUL	MOVF	0x70,W,A	; WREG = [0x370]
	MULWF	0x80	; WREG = [0x380] + WREG
	MOVFF	0xFF3,0x090	; [0x390] = PRODL
	MOVFF	0xFF4,0x091	; [0x391] = PRODH
	RETURN		; Return back to main to branch to end
START	MOVF	00H,0,0	; Get the calculator input choice. ; If the choice is 1 perform addition, for a choice ; 2 perform subtraction, for a choice 3 perform ; multiplication.
	CPFSEQ	01H,0	; Compare to see if choice is 1 or not. ; If not 1 go to check if choice is 2 or 3.
	BRA	L1	; Choice is not 1 so go to check if choice is 2 or 3.
	MOVLB	0x01	; Set bank 1 and perform addition.
	CALL	MY_ADD	; Call Addition module
	BRA	LEND	; Go to end after completion of addition.
	CPFSEQ	02H,0	; Compare to see if choice is 2 or not if its not 1. ; If not 2 go to check if choice is 3.
	BRA	L2	; Choice is not 2 so go to check if choice is 3.
	MOVLB	0x02	; Set bank 2 and perform subtraction.
	CALL	MY_SUB	; Call Subtraction module
L1	BRA	LEND	; Go to end after completion of subtraction.
	CPFSEQ	03H,0	; Compare to see if choice is 3 or not if its not 1,2. ; If not 3 go to end, as its wrong choice.
	BRA	LEND	; Choice is not 3 also, so entered choice is wrong. ; Go to end.
	MOVLB	0x03	; Set bank 3 and perform multiplication.
	CALL	MY_MUL	; Call Multiplication module
L2	NOP		; No Operation.
LEND	NOP		; No Operation.

Result

ADD

0x02 ßFile Register 0x110
0x03 ßFile Register 0x120
0x05 ßFile Register 0x130

MUL

0x02 ßFile Register 0x370
0x03 ßFile Register 0x380
0x06 ßFile Register 0x390 PRODL

SUB

0x02 ßFile Register 0x240
0x03 ßFile Register 0x250
0xFF ßFile Register 0x260

0x00 ßFile Register 0x391 PRODH

Program 5.28 A simple Fast Save based example – Modules: Call(S) & Return(S)

```

MY_ADD1
VAR2    SET      0x20          ; VAR2 = 0x20
        MOVLW    0x18          ; WREG = 0x18
        ADDLW    0x28,0        ; WREG = 0x28 + 0x18 = 0x40;
        ; ACF = 1, STATUS = 0x02
        MOVWF    VAR2,0       ; [0x20] = 0x40
VAR2    SET      0x21          ; VAR2 = 0x21
        MOVFF    STATUS,VAR2   ; [0x21] = STATUS
        RETURN   0 ; 1 or S    ; If RETURN is followed by 0 the WREG,
        ; STATUS and BSR will not be restored.
        ; If 1 or S they are restored.

START
VAR1    SET      0x30          ; VAR1 = 0x30
        MOVLW    0x34          ; WREG = 0x34
        MOVWF    VAR1, 0      ; [0x30] = 0x34
VAR1    SET      0x31          ; VAR1 = 0x31
        MOVFF    STATUS,VAR1   ; ACF = 0, STATUS = 0x00
        ; [0x31] = STATUS
        CALL     MY_ADD1, 0 ;1 or S ; If CALL is followed by 0 the WREG, STATUS
        ; and BSR are not fast saved.
        ; If 1 or S they are saved.
VAR1    SET      0x40          ; VAR1 = 0x40
        MOVWF    VAR1,0       ; [0x40] = WREG = 0x40 if call and
        ; return was done without fast save restoring
        [0x40] = WREG = 0x34 if call and
        ; return was done with fast save restoring
VAR1    SET      0x41          ; VAR1 = 0x41
        MOVFF    STATUS,VAR1   ; ACF = 1, 0x41 = STATUS = 0x02 if call and
        ; return was done without fast save restoring
        ; ACF = 0, 0x41 = STATUS = 0x00 if call and
        ; return was done with fast save restoring
        ; main programs value
        NOP                  ; No Operation

```

S in call saves the WREG, STATUS, BSR registers

S in return restores the WREG, STATUS, BSR registers

S = 0

Address	00	01	02
000	00	00	00
010	00	00	00
020	40	02	00
030	34	00	00
040	40	02	00
050	00	00	00

S = 1

Address	00	01	02
000	00	00	00
010	00	00	00
020	40	02	00
030	34	00	00
040	34	00	00
050	00	00	00

S = S

Address	00	01	02
000	00	00	00
010	00	00	00
020	40	02	00
030	34	00	00
040	34	00	00
050	00	00	00

Macro:

When an instruction group has a smaller number of instructions and needs to be repeated or execute faster without overhead and has no memory restriction they are written as a macro.

Macros are created using assembler directives. A macro has an identification label name is used along with a “MACRO” directive to create/indicate the start of a macro and require a “ENDM” directive at the end of a macro to indicate the end of macro. The macro is invoked by the assembler when it comes across a Macro while assembling and linking before a program is loaded into the program memory of a microcontroller or a microprocessor.

The program written in the editor by the user will certainly not be the same as the one loaded into the program memory of the microcontroller/microprocessor (unless it's optimized by the assembler). It is because instructions written in macro are replicated and are also stored in program memory each time they are required. They are written once in program while entering in editor but the assembler while assembling the program replaces each macro invoking with the complete set of instructions defined as part of a macro in the local program memory.

Since instructions are stored in the memory every time a macro is invoked, jump from main program is not required. This reduces overhead, execution time/steps along with eliminating the need for context saving. Also flushing of the pipeline is not required since jump is not required

The main disadvantage is that the number of memory bytes required is increased and the final program loaded into microcontroller will be the same as if a subroutine/macro is not used at all. As an example if a macro is having 10 instructions and is invoked 20 times, the amount of memory required for 10 instructions, if assumed as 1 word per instruction, requires 200 words (400 bytes) since the same instructions need to be stored in program memory 20 times. This is the same as if it is not written as a subroutine/macro as it requires 200 words (400 bytes). This is the main disadvantage of using macros. The advantage is that the macro needs to be defined only once similar to subroutines and is replicated by the assembler each time when required, without the user/programmer having to do it.

Program 5.29(a) Simple Marcos Example 1

```

MY_1MACRO  MACRO
            MOVF      POSTINC0, 0, 0  ; WREG = [0x110], [0x111]
            ADDWFC     POSTINC1, 0, 0  ; WREG = [0x120] + WREG,
            ; WREG = [0x121] + WREG,
            MOVWF      POSTINC2, 0    ; [0x130] = WREG
            ; [0x131] = WREG
            ENDM
START      LFSR       0, 0x110        ; FSRO = 0x110
            LFSR       1, 0x120        ; FSR1 = 0x120
            LFSR       2, 0x130        ; FSR2 = 0x130
            MY_1MACRO                      ; Macro invoke 1
            MY_1MACRO                      ; Macro invoke 2
            NOP                          ; No Operation

```

The actual code that is generated and loaded into the PIC18F458 is as below.

			Address	HEX Code	Label	Instructions
	ORG	0X30	0030	EE01	START	LFSR 0, 0x110
MY_1MACRO	MACRO		0032	F010		NOP
	MOVF	POSTINC0, 0, 0	0034	EE11		LFSR 1, 0x120
	ADDWFC	POSTINC1, 0, 0	0036	F020		NOP
	MOVWF	POSTINC2, 0	0038	EE21		LFSR 2, 0x130
	ENDM		003A	F030		NOP
START	LFSR	0, 0x110	003C	50EE		MOVF POSTINC0, W, ACCESS
	LFSR	1, 0x120	003E	20E6		ADDWFC POSTINC1, W, ACCESS
	LFSR	2, 0x130	0040	6EDE		MOVWF POSTINC2, ACCESS
	MY_1MACRO		0042	50EE		MOVF POSTINC0, W, ACCESS
	MY_1MACRO		0044	20E6		ADDWFC POSTINC1, W, ACCESS
	MY_1MACRO		0046	6EDE		MOVWF POSTINC2, ACCESS
	NOP		0048	0000		NOP

As can be seen from the figure the macro is stored twice in the memory from 0x3C to 0x41 and 0x42 to 0x47 as the macro MY_1MACRO is invoked twice.

Result

0x03	βFile Register 0x110	0x04	βFile Register 0x111
<u>0x11</u>	βFile Register 0x120	<u>0x22</u>	βFile Register 0x121
0x14	βFile Register 0x130	0x26	βFile Register 0x131

Program 5.29(b) Marcos Example with parameter passing.

```

MY_1MACRO  MACRO      K, MY_REG      ; Define Macro
            MOVLW      K              ; WREG = k
            ADDWF      MY_REG,0,0     ; WREG = MY_REG+WREG
            ENDM                      ; End Macro
START      NOP                      ; No Operation
            MY_1MACRO  0x11, 0x20     ; Macro invoke 1
            MY_1MACRO  0x22, 0x21     ; Macro invoke 2
            MY_1MACRO  0x33, 0x22     ; Macro invoke 3
            MY_1MACRO  0x44, 0x23     ; Macro invoke 4
            NOP                      ; No Operation

```

Result

0x02	βFile Register 0x20	0x03	βFile Register 0x21
<u>0x11</u>		<u>0x22</u>	
0x13	βFile Register 0x20	0x25	βFile Register 0x21
0x04	βFile Register 0x22	0x05	βFile Register 0x23
<u>0x33</u>		<u>0x44</u>	
0x37	βFile Register 0x22	0x49	βFile Register 0x23

Program 5.30 Marcos with a Local Variable L1 for ‘Times’ number of loops

```

MY_MDELAY  MACRO      TIMES,          ; Macro Definition
            MY_LOC      MY_LOC
            LOCAL      L1              ; Declaring Local label L1
            MOVLW      TIMES           ; WREG = Times
            MOVWF      MY_LOC          ; MY_LOC = TIMES
L1          NOP                      ; No Operation
            NOP                      ; No Operation
            DECF      MY_LOC,F         ; Reduces Times value till zero.
            BNZ      L1                ; If zero exit macro.
            ENDM                      ; End of macro
START      NOP                      ; No Operation
            MY_MDELAY  0x05, 0x20     ; Macro Invoke 1
            MY_MDELAY  0x08, 0x21     ; Macro Invoke 2
            NOP                      ; No Operation

```

			Address	HEX Code	Label	Instructions
	ORG	0X30	0030	EE01	START	LFSR 0, 0x110
MY_MACRO	MACRO		0032	F010		NOP
	MOVWF	POSTINC0, 0, 0	0034	EE11		LFSR 1, 0x120
	ADDWFC	POSTINC1, 0, 0	0036	F020		NOP
	MOVWF	POSTINC2, 0	0038	EE21		LFSR 2, 0x130
	ENDM		003A	F030		NOP
START	LFSR	0, 0x110	003C	50EE		MOVF POSTINC0, W, ACCESS
	LFSR	1, 0x120	003E	20E6		ADDWFC POSTINC1, W, ACCESS
	LFSR	2, 0x130	0040	6EDE		MOVWF POSTINC2, ACCESS
	MY_MACRO		0042	50EE		MOVF POSTINC0, W, ACCESS
	MY_MACRO		0044	20E6		ADDWFC POSTINC1, W, ACCESS
	MY_MACRO		0046	6EDE		MOVWF POSTINC2, ACCESS
	NOP		0048	0000		NOP

Reproduced Program 5.29(a) Simple macro example reproduced here for comparison.

Difference between subroutines and macro

As can be seen from the subroutine and macro in program 5.25(b) and 5.29(a), if the number of instructions in the group and number of invokes is less, macros are better and require less memory. Here neglecting the final NOP in the program, subroutine used 28 (0x30 to 0x4B) locations whereas macros used 24 (0x30 to 0x47) locations. Also, the execution time is more in subroutines as it additionally executes a call and return instruction every time.

			Address	HEX Code	Label	Instructions
	ORG	0X30	0030	50EE		MOVF POSTINC0, W, ACCESS
MY_SR	MOVWF	POSTINC0, 0, 0	0032	20E6		ADDWFC POSTINC1, W, ACCESS
	ADDWFC	POSTINC1, 0, 0	0034	6EDE		MOVWF POSTINC2, ACCESS
	MOVWF	POSTINC2, 0	0036	0012		RETURN 0
	RETURN		0038	EE01		LFSR 0, 0x110
			003A	F010		NOP
			003C	EE11		LFSR 1, 0x120
			003E	F020		NOP
START	LFSR	0, 0x110	0040	EE21		LFSR 2, 0x130
	LFSR	1, 0x120	0042	F030		NOP
	LFSR	2, 0x130	0044	EC18		CALL 0x30, 0
	CALL	MY_SR	0046	F000		NOP
	CALL	MY_SR	0048	EC18		CALL 0x30, 0
	NOP		004A	F000		NOP

The example programs 5.31(a) and 5.31(b) is an extended version of program 5.25(b) and 5.29(a). This example is purely used to demonstrate difference between macro and subroutine, and the logic of the program is not the essential criteria. So, three more instructions are added in the group along with an additional invoke. This increases the number of bytes used by macros to be higher than that of subroutines.

Here neglecting the final NOP in the program, subroutines used 38 (0x30 to 0x55) locations whereas macros used 48 (0x30 to 0x5F) locations even though subroutine had extra 2 instructions - Call and Return, both being executed every time a subroutine is called. The

execution time is more for subroutines always as it additionally executes a call and return instruction every time compared to a macro. The demonstration of execution time taken is presented in timers' section where the number of cycles taken for execution is considered.

Reproduced Program 5.25(b) Simple subroutine example reproduced here for comparison.

	Address	HEX Code	Label	Instructions
	0030	EE01		LFSR 0, 0x110
	0032	F010		NOP
	0034	EE11		LFSR 1, 0x120
	0036	F020		NOP
	0038	EE21		LFSR 2, 0x130
	003A	F030		NOP
MY_1MACRO	003C	50EE		MOVF POSTINC0, W, ACCESS
	003E	20E6		ADDWFC POSTINC1, W, ACCESS
	0040	6EDE		MOVWF POSTINC2, ACCESS
	0042	50EE		MOVF POSTINC0, W, ACCESS
	0044	5CE6		SUBWF POSTINC1, W, ACCESS
	0046	6EDE		MOVWF POSTINC2, ACCESS
	0048	50EE		MOVF POSTINC0, W, ACCESS
	004A	20E6		ADDWFC POSTINC1, W, ACCESS
	004C	6EDE		MOVWF POSTINC2, ACCESS
	004E	50EE		MOVF POSTINC0, W, ACCESS
	0050	5CE6		SUBWF POSTINC1, W, ACCESS
	0052	6EDE		MOVWF POSTINC2, ACCESS
	0054	50EE		MOVF POSTINC0, W, ACCESS
	0056	20E6		ADDWFC POSTINC1, W, ACCESS
	0058	6EDE		MOVWF POSTINC2, ACCESS
MY_1MACRO	005A	50EE		MOVF POSTINC0, W, ACCESS
MY_1MACRO	005C	5CE6		SUBWF POSTINC1, W, ACCESS
MY_1MACRO	005E	6EDE		MOVWF POSTINC2, ACCESS
	0060	0000		NOP
START				
				LFSR 0, 0x110
				LFSR 1, 0x120
				LFSR 2, 0x130
				ENDM

Program 5.31(a) Extended version of program 5.25(b) for comparison.

	Address	HEX Code	Label	Instructions
MY_SR	0030	50EE		MOVF POSTINC0, W, ACCESS
	0032	20E6		ADDWFC POSTINC1, W, ACCESS
	0034	6EDE		MOVWF POSTINC2, ACCESS
	0036	50EE		MOVF POSTINC0, W, ACCESS
	0038	5CE6		SUBWF POSTINC1, W, ACCESS
	003A	6EDE		MOVWF POSTINC2, ACCESS
	003C	0012		RETURN 0
	003E	EE01		LFSR 0, 0x110
	0040	F010		NOP
	0042	EE11		LFSR 1, 0x120
	0044	F020		NOP
	0046	EE21		LFSR 2, 0x130
	0048	F030		NOP
	004A	EC18		CALL 0x30, 0
	004C	F000		NOP
	004E	EC18		CALL 0x30, 0
	0050	F000		NOP
	0052	EC18		CALL 0x30, 0
	0054	F000		NOP
START				
				LFSR 0, 0x110
				LFSR 1, 0x120
				LFSR 2, 0x130
				CALL MY_SR
				CALL MY_SR
				CALL MY_SR
				NOP

Program 5.31(b) Extended version of program 5.29(a) for comparison.

So, subroutines are preferred when memory required is limited and macros are preferred when execution time is of importance.

5.2 C Language Programming Examples

The sample programs presented in assembly language in section 5.1 are translated to equivalent C programs here.

Program 5.1(a) 8-Bit Addition using Immediate Addressing

```
#include <xc.h>
void main(void)
{
    unsigned char W;
    W = 0x34;
    W = W + 0x23;
    *(volatile unsigned char *)0x30 = W;
    while(1)
    { // Equivalent to program end }
}
```

Program 5.1(b) 8-Bit Addition using Direct Addressing

```
#include <xc.h>
void main(void)
{
    unsigned char result;
    result = *(volatile unsigned char *)0x10;
    result += *(volatile unsigned char *)0x20;
    *(volatile unsigned char *)0x30 = result;
    while(1)
    { // Equivalent to program end }
}
```

Program 5.1(c) 8-Bit Addition using Indirect Addressing

```
#include <xc.h>
void main(void)
{
    unsigned char *FSR0 = (unsigned char *)0x110;
    unsigned char *FSR1 = (unsigned char *)0x120;
    unsigned char *FSR2 = (unsigned char *)0x130;
    unsigned char WREG; WREG = *FSR0++;
    WREG = WREG + *FSR1++;
    *FSR2 = WREG;
    while(1)
```

```

        { // Equivalent to program end }
    }

```

Program 5.2(a) Addition of 16-Bit number using direct addressing mode.

```

#include <xc.h>
void main(void)
{
    unsigned char low, high;
    unsigned int temp;

    /* Add low bytes /
    temp = (unsigned int)(unsigned char )0x10 + (unsigned int)(unsigned char *)0x20;
    *(unsigned char )0x30 = (unsigned char) temp;

    / Add high bytes with carry */
    high = *(unsigned char *)0x11 + *(unsigned char *)0x21 + ((temp > 0xFF) ? 1 : 0);
    *(unsigned char *)0x31 = high;
    while(1)
    { // Equivalent to program end }

}

```

Program 5.2(b) Addition of 16-Bit number using indirect addressing modes.

```

#include <xc.h>
void main(void)
{
    uint8_t *fsr0 = (uint8_t *)0x110;
    uint8_t *fsr1 = (uint8_t *)0x120;
    uint8_t *fsr2 = (uint8_t *)0x130;
    uint16_t temp;

    / Low-byte addition */
    temp = *fsr0++ + *fsr1++; fsr2++ = (uint8_t)temp;

    / High-byte addition with carry */
    temp = *fsr0++ + *fsr1++ + (temp >> 8);
    *fsr2++ = (uint8_t)temp;
    while(1)
    { // Equivalent to program end }

}

```

Program 5.3 Subtraction of two 8-Bit in immediate and 16-Bit numbers in direct mode.

Direct Addressing


```

#include <xc.h>
void main(void)
{
    uint8_t W;
    W = 0x23;
    W = 0x34 - W;
    memory[0x30] = W;
    while(1)
    { // Equivalent to program end }
}

```

Indirect Addressing

```

#include <xc.h>
void main(void)
{
    unsigned char mem[256];
    /* 16-bit subtraction:
       result = value_at_0x20_21 - value_at_0x10_11
    */
    unsigned int a = ((unsigned int)mem[0x21] << 8) | mem[0x20];
    unsigned int b = ((unsigned int)mem[0x11] << 8) | mem[0x10];
    unsigned int result = a - b;
    mem[0x30] = result & 0xFF;    // low byte
    mem[0x31] = (result >> 8) & 0xFF; // high byte
    while(1)
    { // Equivalent to program end }
}

```

Program 5.4 Demonstration of flags & their status update

```

#include <xc.h>
void main(void)
{
    unsigned char W; unsigned char mem[256];
    /* First sequence */
    W = 0x01;
    W = W + 0x02;
    mem[0x30] = W; // mem[0x30] = 0x03

    /* Second sequence */
    W = 0x01;

```

```

W = 0xFF - W;    // SUBLW performs literal - W
mem[0x30] = W; // mem[0x30] = 0xFE
while(1)
{ // Equivalent to program end }
}

```

Program 5.5 Multiplication of two 8-Bit numbers.

```

#include <xc.h>
void main(void)
{
    unsigned char mem[256];
    /* simulate indirect pointers */
    unsigned int FSR0 = 0x110;
    unsigned int FSR1 = 0x120;
    unsigned int FSR2 = 0x130;
    /* POSTINC0 / POSTINC1 behavior */
    unsigned char A = mem[FSR0++]; // mem[0x110]
    unsigned char B = mem[FSR1++]; // mem[0x120]
    unsigned int product = (unsigned int)A * (unsigned int)B;
    /* store result using POSTINC2 */
    mem[FSR2++] = product & 0xFF; // low byte → 0x130
    mem[FSR2++] = (product >> 8) & 0xFF; // high byte → 0x131
    while(1)
    { // Equivalent to program end }
}

```

Program 5.6 If Condition

The program performs an equivalence of the following conditional if statement in assembly language.

```

if (x<5)
{
    y=y+1;
    z=z+1;
    x=x+1;
}

```

Program

```

#include <xc.h>
void main(void)
{
    unsigned char mem[256];

```

```

    unsigned char W = mem[0x11];
    /* CPFSLT behavior: skip next instruction if W < mem[0x10] /
    if (W < mem[0x10])
    {      / next instruction (BRA L1) is skipped → execute block */
        mem[0x13]++;
        mem[0x14]++;
        mem[0x10]++;
    }
    while(1)
    { // Equivalent to program end }
}

```

Program 5.7 While Condition

The program performs an equivalence of the following while loop in assembly language.

```

while (x<5)
{
    y=y+1;
    z=z+1;
    x=x+1;
}

```

Program

```

#include <stdint.h>
void program(volatile uint8_t *mem)
{
    uint8_t WREG;
    WREG = mem[0x11]; // MOVF 0x11,0
    while (mem[0x10] < WREG) // CPFSLT logic inverted into loop condition
    {
        mem[0x13]++; // INCF 0x13
        mem[0x14]++; // INCF 0x14
        mem[0x10]++; // INCF 0x10,1
    }
}

```

Program 5.8 Multi-Byte addition (32-Bit Addition: Four 8-Bit Additions)

```

#include <stdint.h>

```

```

int main(void)
{
    // Simulate PIC data memory as arrays/pointers
    uint8_t *FSR0 = (uint8_t *)0x110;
    uint8_t *FSR1 = (uint8_t *)0x120;
    uint8_t *FSR2 = (uint8_t *)0x130;
    uint8_t count = 4;
    uint8_t carry = 0;

    while (count-- > 0)
    {
        uint16_t temp;

        // WREG = POSTINC0
        temp = *FSR0++;

        // ADDWFC POSTINC1, W (W + *FSR1 + carry)
        temp = temp + *FSR1++ + carry;

        // store result to POSTINC2
        *FSR2++ = temp;

        // update carry (important for ADDWFC behavior)
        carry = (temp > 0xFF) ? 1 : 0;
    }
    return 0;
}

```

Program 5.9 Block Data Transfer

The program performs multiple data transfers, or the data transfer of a block. This is a modified version of a multi-byte addition program where FSR1 is not used, and addition is not performed. Instead, data pointed by FSR0 is transferred to locations pointed by FSR2. Here, the ALP is coded to transfer 32-Bits or 4 bytes from file registers (pointed by FSR0) 0x110, 0x111, 0x112, 0x113 to file registers (pointed by FSR2) 0x120, 0x121, 0x122, 0x123.

```

#include <stdint.h>
void copy_block(uint8_t *src, uint8_t *dst, uint8_t count)
{
    while (count != 0)
    {
        *dst++ = *src++;
        count--;
    }
}

```

```

    }
}

```

Program 5.10 (a) Division by a factor of 2 using Rotate Instruction

The program performs division by using Rotate RRCF instruction using direct addressing mode. The 8-Bit data (0x42) from file register 0x30 is read and divided by 2 using RRCF. The result 0x21 is stored in file register 0x31.

```

#include <stdint.h>
/* Simulated memory-mapped registers */
volatile uint8_t * const REG30 = (uint8_t *)0x30;
volatile uint8_t * const REG31 = (uint8_t *)0x31;
/* STATUS carry flag (bit 0) */
uint8_t C = 0;
void main(void)
{
    uint8_t W;
    /*          MOVF          0x30,          W          */
    W          =          *REG30;

    /*          BCF          STATUS,          0          */
    C          =          0;

    /*          RRCF          0x30,          W          */
    /* Rotate          right          through          carry: */
    /* result          =          (value          >>          1)          |          (C          <<          7) */
    /*          */
    W          =          (*REG30          >>          1)          |          (C          <<          7);

    /*          MOVWF          0x31          */
    *REG31 = W;
}

```

Program 5.10 (b) Division by a factor of 16 using Rotate Instruction.

The program performs division of a number by 16 using Rotate RRCF instruction using direct addressing mode. The 8-Bit data (0x42) from file register 0x30 is read and divided by 16 using RRCF in a loop. The result 0x04 is stored in file register 0x30.

```

#include <stdint.h>
uint8_t mem[256];          // simulate PIC data memory (optional model)
void pic18_emulation(void)
{

```

```

uint8_t *reg30 = &mem[0x30];
uint8_t counter = 0x04;
// MOVF 0x30, W is in original code but has no effect afterward

while (counter != 0)

{
    //      BCF      STATUS,    0      ->    clear    carry
    uint8_t      carry_in      =      0;

    //      RRCF      0x30,F      with      carry      cleared
    uint8_t new_carry = (*reg30) & 0x01; // bit shifted out
    *reg30 = (carry_in << 7) | (*reg30 >> 1);

    //      DECF      0x10,      F
    counter--;
}
}

```

Program 5.12 Division of 8-Bit/8-Bit numbers using repetitive subtraction

The program performs division of two 8-Bit numbers using direct addressing mode. It uses repeated subtraction technique, as no DIV instruction is available in PIC18F458 and PIC16F877A. The 8-Bit data (0x15) and (0x02) are read from file registers 0x10 & 0x20 and repeated subtraction is performed through a loop. The quotient 0x0A remainder 0x01 is stored in file register 0x30 and 0x31 respectively.

```

#include <stdint.h>
uint8_t mem[256]; // simulate data memory
void program(void)
{
    mem[0x30] = mem[0x10]; // MOVF 0x10,0 / MOVWF 0x30
    while (1)
    {
        uint8_t W = mem[0x20]; // MOVF 0x20,0

        // CPFSGT 0x30,0
        // skip next instruction if mem[0x30] > W
        if (mem[0x30] <= W)
        {

```

```

        break; // BRA L1
    }

    // SUBWF 0x30,1,0 => 0x30 = 0x30 - W
    mem[0x30] = mem[0x30] - W;

    // INCF 0x31,1,0
    mem[0x31]++;
}
}

```

Program 5.13(a) Multiplication of 16-Bit x 8-bit number to produce a 24-Bit result.

The program performs multiplication of a 16-Bit number with an 8-Bit number using direct addressing mode. It is an extension of program 5.11. The 16-Bit data (0x70 0x80) are read from file registers 0x11 & 0x10 and the 8-Bit data from 0x20. The result 0x3F 0x48 0x00 is stored in file register 0x32, 0x31 and 0x30 respectively.

```

#include <stdint.h>
#include <stdbool.h>

uint8_t mem[256];    // Represents PIC file registers
uint8_t W;           // Working register
bool C;              // Carry flag

void program(void)
{
    // MOVF 0x20, 0 -> W = mem[0x20]
    W = mem[0x20];

    // MOVWF 0x40 -> mem[0x40] = W
    mem[0x40] = W;

L2:
    // MOVF 0x10, 0
    W = mem[0x10];

    // ADDWF 0x30, 1, 0 -> mem[0x30] += W

```

```

uint16_t temp1 = (uint16_t)mem[0x30] + W;
mem[0x30] = (uint8_t)temp1;

// MOVF 0x11, 0
W = mem[0x11];

// ADDWFC 0x31, 1, 0 -> mem[0x31] += W + C
uint16_t temp2 = (uint16_t)mem[0x31] + W + (C ? 1 : 0);
mem[0x31] = (uint8_t)temp2;

// Update carry from second addition
C = (temp2 > 0xFF);

// BNC L1 -> if no carry, skip increment of 0x32
if (C)
{
    // INCF 0x32, 1, 0
    mem[0x32]++;
}

L1: // DECF 0x40, 1, 0
    mem[0x40]--;

    // BNZ L2 -> loop while mem[0x40] != 0
    if (mem[0x40] != 0)
        goto L2;
}

```

Program 5.13(b) Division of 16-Bit/8-Bit numbers using repetitive subtraction

The program performs division of a 16-Bit number by an 8-Bit number using direct addressing mode. It is an extension of program 5.12.

```
#include <stdint.h>
```

```

// emulate PIC file registers
uint8_t REG[256];

```



```

void program(void)
{
    uint8_t W;

    // MOVF 0x10 -> 0x30
    W = REG[0x10];
    REG[0x30] = W;

    // MOVF 0x11 -> 0x31
    W = REG[0x11];
    REG[0x31] = W;

L2:
    // W = REG[0x20]
    W = REG[0x20];

    // CPFSLT 0x30,0
    // skip next instruction if REG[0x30] < W
    if (REG[0x30] < W)
        goto LN;
    else
        goto L1;

L1:
    // REG[0x30] = REG[0x30] - W
    {
        uint16_t result = (uint16_t) REG[0x30] - (uint16_t) W;

        // emulate borrow behavior
        uint8_t no_borrow = (REG[0x30] >= W);

        REG[0x30] = (uint8_t) result;

        // INCF 0x32
        REG[0x32]++;

        // BNC (branch if no carry => no borrow = true carry flag in subtraction)

```

```

        if (!no_borrow)
        {
            // borrow occurred -> skip increment 0x33
        }
        else
        {
            REG[0x33]++;
        }
    }
    goto L2;
LN:
    // MOVLW 0x00
    W = 0x00;
    // CPFSGT 0x31,0
    // skip next if REG[0x31] > 0
    if (REG[0x31] > 0)
        goto LN_DEC;
    else
        goto L3;
LN_DEC:
    // DECF 0x31
    REG[0x31]--;
    goto L1;
L3:
    return;
}

```

Program 5.14 Packed BCD to Unpacked & then to ASCII

The program performs conversion of a packed BCD number to unpacked numbers and then to ASCII numbers.

```

#include <stdint.h>

uint8_t reg10 = 0x10;    // input register (0x10 in ASM)
uint8_t reg11, reg12;
uint8_t reg21, reg22;

void main(void)

```

```

{
    uint8_t temp;

    // MOVF 0x10, W
    temp = reg10;

    // ANDLW 0x0F
    reg11 = temp & 0x0F;

    // MOVWF 0x21
    reg21 = reg11;

    // MOVF 0x10, W
    temp = reg10;

    // ANDLW 0xF0
    reg12 = temp & 0xF0;

    // SWAPF 0x12, 1
    reg12 = reg12 >> 4;    // equivalent to swapping upper nibble to lower

    // MOVFF 0x12, 0x22
    reg22 = reg12;

    // MOVLW 0x30
    // ADDWF 0x21, 1
    reg21 += 0x30;

    // ADDWF 0x22, 1
    reg22 += 0x30;
}

```

Program 5.15 ASCII to Unpacked BCD & then to Packed

The program performs conversion of group ASCII numbers to unpacked numbers and then to a packed BCD number.

```
#include <stdint.h>
```

```

// Simulated RAM locations (file registers)
uint8_t mem[0x100]; // 0x00–0xFF PIC-style data space
void main(void)
{
    uint8_t W;

    // MOVF 0x10, W
    W = mem[0x10];

    // ANDLW 0x0F
    W &= 0x0F;

    // MOVWF 0x21
    mem[0x21] = W;

    // MOVF 0x11, W
    W = mem[0x11];

    // ANDLW 0x0F
    W &= 0x0F;

    // MOVWF 0x22
    mem[0x22] = W;

    // MOVWF 0x30 (same W still holds low nibble of 0x11)
    mem[0x30] = W;

    // SWAPF 0x30, 1 (swap nibbles in file register 0x30)
    mem[0x30] = (uint8_t) ((mem[0x30] << 4) | (mem[0x30] >> 4));

    // MOVF 0x11, W
    W = mem[0x11];

    // ADDWF 0x30, 1
    mem[0x30] = mem[0x30] + W;
}

```

Program 5.16 Number of 1's & 0's in a byte

The program counts the number of 1's and 0's present in a 8-Bit number. Such a count is helpful to determine the number of Input/Output - I/O's in ON state or the OFF state. Ex: Reading the switch status and determine the number of switches in ON or OFF position.

```
#include <stdint.h>
int main()
{
    uint8_t reg10 = 0x08;
    uint8_t reg11 = 0x08;
    uint8_t reg20 = 0x00; // assume some initial value
    uint8_t reg21 = reg20;
    uint8_t reg30 = 0;
    uint8_t reg31 = 0;

    while (reg11 != 0)
    {
        // Rotate reg21 left through carry
        uint8_t carry = (reg21 & 0x80) >> 7;
        reg21 = (reg21 << 1) | carry;

        if (carry)
        {
            reg30++;
        }
        else
        {
            reg31++;
        }

        reg11--; // DECF 0x11,F
    }
    return 0;
}
```

Program 5.17 Bit Set/Reset: Byte Wise

The purpose of this program is to demonstrate bit manipulation. Flags operations are common cases of bit manipulations. The other main usage of bit manipulations is for setting and resetting of peripheral control bits or I/O's. This program mainly demonstrates performing of bit manipulation on a byte of data to mimic cases where bit manipulation instructions as such are not present i.e like 8086 which only have flag-based bit manipulations. Also, such logic helps in changing bit value in a byte when data can be read or written only as a byte like memory devices etc.

Here N1, N2, N3 & N4 are considered four memory locations values (numbers) that are read or written as a byte. B0 to B7 are the bit positions in a byte. N1B1 indicates Bit 1 in number 1, N4B5 indicates bit 5 in number 4 and so on.

```
if (N1B1 ==0)    {N3B3=0;}
else            {N3B3=1;}
if(N2B2==1)     {N4B5=0;}
else            {N4B5=1;}
```

```
#include <stdint.h>
```

```
int main()
```

```
{
```

```
    // Copy registers
```

```
    REG20 = REG30;
```

```
    REG21 = REG31;
```

```
    // Modify REG30 based on bit 1 of REG10
```

```
    if (REG10 & 0x02)
```

```
    {
```

```
        REG30 |= 0x08; // set bit 3
```

```
    }
```

```
    else
```

```
    {
```

```
        REG30 &= 0xF7; // clear bit 3
```

```
    }
```

```
    // Modify REG31 based on bit 2 of REG11
```

```
    if (REG11 & 0x04)
```

```
    {
```

```
        REG31 &= 0xDF; // clear bit 5
```

```
    }
```

```

        else
        {
            REG31 |= 0x20; // set bit 5
        }
    return 0;
}

```

Program 5.18 Bit Set/Reset: Bit Wise

The purpose of this program is to demonstrate bit manipulation using bit-wise instructions. This logic helps in changing bit values in a byte when data can be read or written only as a byte like memory devices etc.

Here N1, N2, N3 & N4 are considered four memory locations values (numbers) that are read or written as a byte. B0 to B7 are the bit positions in a byte. N1B1 indicates Bit 1 in number 1, N4B5 indicates bit 5 in number 4 and so on.

```

if (N1B1 == 0)    {N3B3=0;}
else              {N3B3=1;}
if(N2B2==1)      {N4B5=0;}
else              {N4B5=1;}

```

```

#include <stdint.h>
unsigned char reg10, reg11; // Corresponds to 0x10, 0x11
unsigned char reg30, reg31; // Corresponds to 0x30, 0x31
unsigned char reg20, reg21; // Corresponds to 0x20, 0x21
void asm_equivalent()
{
    // Copy registers
    reg20 = reg30;
    reg21 = reg31;

    // Process bit 1 of reg10
    if (reg10 & (1 << 1))
    {
        reg30 |= (1 << 3); // set bit 3
    }
    else
    {

```

```

        reg30 &= ~(1 << 3); // clear bit 3
    }

    // Process bit 2 of reg11
    if (reg11 & (1 << 2))
    {
        reg31 &= ~(1 << 5); // clear bit 5
    }
    else
    {
        reg31 |= (1 << 5); // set bit 5
    }
    // NOP - nothing to do
    return 0;
}

```

Program 5.19 Minimum/Maximum of given set of 8-Bit numbers

The program finds the maximum of a given set of 8-bit numbers starting from file register 0x20.

```

#include <stdint.h>
void asm_to_c(uint8_t *mem)
{
    uint8_t *fsr0 = &mem[0x20]; // LFSR 0, 0x20

    uint8_t W = 0x04;           // MOVLW 0x04
    mem[0x10] = W;               // MOVWF 0x10
    mem[0x11] = W;               // MOVWF 0x11 (loop counter)

    W = *fsr0++;                 // MOVF POSTINC0,0

    uint8_t count = mem[0x11];

    while (count--)
    {
        uint8_t current = *fsr0; // INDF0
        if (W < current)

```



```

        {
            mem[0x12] = W;    // CPFSLT path
        }
        else
        {
            mem[0x12] = current; // branch L3
        }
        W = *fsr0++;        // MOVF POSTINC0,1
    }
return 0;
}

```

Program 5.20 Sorting in Ascending/Descending Order of a given set of 8-Bit numbers

The program finds the ascending of a given set of 8-bit numbers starting from file register 0x20.

```

#include <stdint.h>
void asm_to_c_translation()
{
    uint8_t array[4] = {0, 0, 0, 0}; // placeholder, original memory
    uint8_t reg10 = 4;
    uint8_t reg12 = 3;
    uint8_t reg11, tmp;
    while (reg12 != 0)
    {
        reg11 = reg12;
        uint8_t* ptr = array; // pointer to start of array

        while (reg11 != 0)
        {
            if (*ptr > *ptr)
            {
                // corresponds to CPFSLT, need more context
                tmp = *ptr;
                *ptr = *ptr; // swapping logic from ASM
                *ptr = tmp;
            }
            ptr++;
            reg11--;
        }
        reg12--;
    }
}

```

```
}
```

Program 5.21 Branch Backwards Coding Example: Finding Offsets/Relative Value

The purpose of this program is to demonstrate the calculation of Relative/offset value during a backward jump. This logic helps in understanding hex codes, bytes used for instructions, change in PC value and calculation of relative value etc that may be used to decide whether to use a branch (11-Bit jump) or goto (20-Bit jump) instruction. i.e 2K jump for branch and 1MB or anywhere jump for goto.

```
#include <stdint.h>
int main(void)
{
    uint8_t reg0 = 0x02; // 0x00
    uint8_t reg1 = 0x00; // 0x01
    uint8_t reg3 = 0x00; // 0x03 (never reached)

    while (1)
    {
        reg1 = 0x04;
        uint8_t w = reg1 + reg0;

        // PIC18 BNC checks carry flag; for 8-bit addition:
        // carry occurs if result > 255
        uint8_t carry = (w < reg1); // or (reg1 + reg0 > 0xFF)

        if (!carry)
        {
            continue; // corresponds to BNC L1
        }
        break; // would exit loop if carry occurred (it never does here)
    }
    reg3 = 0x07; // unreachable in this program
    return 0;
}
```

Program 5.22 Branch Forward Coding Example: Finding Offsets/Relative Value

The purpose of this program is to demonstrate the calculation of Relative/offset value during a forward jump. This logic helps in understanding hex codes, bytes used for instructions, change in PC value and calculation of relative value etc that may be used to decide whether to use a branch (11-Bit jump) or goto (20-Bit jump) instruction. i.e 2K jump for branch and 1MB or anywhere jump for goto.

```
#include <stdint.h>

int main()
{
    uint8_t WREG;

    WREG = 0x02;  // MOVLW 0x02
    REG0 = WREG;  // MOVWF 0x00
    WREG += 0x05; // ADDLW 0x05

    if (WREG < 0x100)
    {
        // BNC = branch if no carry (sum < 256)
        WREG = 0x07; // MOVLW 0x07
    }
    else
    {
        WREG = 0x04; // MOVLW 0x04
        REG1 = WREG; // MOVWF 0x01
        WREG = 0x07; // MOVLW 0x07
    }
    REG3 = WREG; // MOVWF 0x03
    return 0;   // NOP
}
```

Program 5.22 Branch Forward Coding Example: Finding Offsets/Relative Value

The purpose of this program is to demonstrate the calculation of Relative/offset value during a forward jump. This logic helps in understanding hex codes, bytes used for instructions, change in PC value and calculation of relative value etc that may be used to decide whether to use a branch (11-Bit jump) or goto (20-Bit jump) instruction. i.e 2K jump for branch and 1MB or anywhere jump for goto.

```
#include <stdint.h>
```

```

int main()
{
    uint8_t REG0 = 0; // corresponds to 0x00
    uint8_t REG1 = 0; // corresponds to 0x01
    uint8_t REG3 = 0; // corresponds to 0x03
    uint8_t W;        // W register

    W = 0x02;         // MOVLW 0x02
    REG0 = W;          // MOVWF 0x00

    W = W + 0x05;      // ADDLW 0x05

    if (W & 0x100)
    {
        // Check carry (more than 8 bits)
        W = 0x04;
        REG1 = W;
    }

    W = 0x07;         // MOVLW 0x07
    REG3 = W;          // MOVWF 0x03

    // NOP does nothing
    return 0;
}

```

Program 5.23(a) Multi Bank 8-Bit Addition

The purpose of this program is to demonstrate how data can be accessed from different banks.

A simple 8-bit addition program discussed earlier is used for this purpose considering data in different banks. Here each 8-bit data required to perform addition are in bank 0 and bank 1 with results in bank 2. i.e file register 0x10, 0x110 and 0x210 respectively.

```

#include <stdint.h>
// Variables representing registers in different banks
unsigned char reg10_bank0;
unsigned char reg10_bank1;
unsigned char reg10_bank2;

```

```

void main()
{
    unsigned char WREG;

    // Equivalent of MOVF 0x10,0,0 (Bank 0)
    WREG = reg10_bank0;

    // Equivalent of ADDWF 0x10,0,1 (Bank 1)
    WREG = WREG + reg10_bank1;

    // Equivalent of MOVWF 0x10,1 (Bank 2)
    reg10_bank2 = WREG;
}

```

Program 5.23(b) Multi Bank 16-Bit Addition

In this program, each 16-bit data required to perform addition is in bank 0 and bank 1 with results in bank 2. i.e. file register 0x11, 0x10 containing 16-bit number1, 0x111, 0x110 containing 16-bit number 2 and 0x212, 0x211 containing the 16-bit result respectively.

```

#include <stdint.h>
// Simulated PIC RAM: 4 banks (0–3), 256 bytes each
uint8_t RAM[4][256];

// Simulated CPU state
uint8_t W = 0;
uint8_t C = 0; // Carry flag

void program(void)
{
    uint8_t bank;

    // --- MOVLB 0x0
    bank = 0;

    // MOVF 0x10,0,0 -> W = RAM[0][0x10]
    W = RAM[0][0x10];
}

```

```

// MOVLB 0x1
bank = 1;

// ADDWF 0x10,0,1 -> W = W + RAM[1][0x10]
{
    uint16_t result = (uint16_t)W + RAM[1][0x10];
    W = (uint8_t)result;
    C = (result > 0xFF);
}

// MOVLB 0x2
bank = 2;

// MOVWF 0x10,1 -> RAM[2][0x10] = W
RAM[2][0x10] = W;

// --- second sequence ---

// MOVLB 0x0
bank = 0;

// MOVF 0x11,0,0 -> W = RAM[0][0x11]
W = RAM[0][0x11];

// MOVLB 0x1
bank = 1;

// ADDWFC 0x11,0,1 -> W = W + RAM[1][0x11] + C
{
    uint16_t result = (uint16_t)W + RAM[1][0x11] + C;
    W = (uint8_t)result;
    C = (result > 0xFF);
}

// MOVLB 0x2
bank = 2;

```

```

// MOVWF 0x11,1 -> RAM[2][0x11] = W
RAM[2][0x11] = W;
}

```

Program 5.24 Accessing SFR's across banks in direct addressing mode.

The purpose of this program is to demonstrate accessing SFR's across banks. Here the PORTB SFR is used for this purpose whose address is 0xF81 and the address of WREG is 0xFE8.

```

#include <xc.h> // XC8 compiler header for PIC18F458
void main(void)
{
    // Set PORTB as output
    TRISB = 0x00;

    // Variables for special registers (addresses 0x81 and 0xFE8)
    volatile unsigned char reg_0x81_bank0;
    volatile unsigned char reg_0x81_bank1;
    volatile unsigned char reg_0xFE8;

    // Equivalent of MOVLW 0x05; MOVWF 0x81,1 and 0x81,0
    reg_0x81_bank1 = 0x05;
    reg_0x81_bank0 = 0x05;

    // MOVLW 0x09
    unsigned char temp = 0x09;

    // MOVFF 0xFE8, 0xF81
    volatile unsigned char reg_0xF81;
    reg_0xF81 = reg_0xFE8;

    // Set PORTB values
    PORTB = 0x06; // Low byte
    // If PIC18F458 PORTB is only 8-bit, we ignore high byte
    // Otherwise, you may need to use LATB or split handling for high byte

```

```

// Here, assuming simple 8-bit operation:
PORTB = 0x07; // High byte / new value

// No operation (NOP)
__nop(); // XC8 intrinsic
}

```

Program 5.25(a) A simple example of subroutine that performs 8-Bit Addition, (without passing of parameters)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.1(b).

```

#include <stdint.h>
// Simulate PIC data memory locations
volatile uint8_t * const REG10 = (uint8_t *)0x10;
volatile uint8_t * const REG20 = (uint8_t *)0x20;
volatile uint8_t * const REG30 = (uint8_t *)0x30;
uint8_t MY_ADD(void)
{
    uint8_t w;
    w = *REG10;    // MOVF 0x10,0,0
    w = w + *REG20; // ADDWF 0x20,0,0
    *REG30 = w;    // MOVWF 0x30,0
    return w;      // equivalent of leaving result in WREG
}
void START(void)
{
    MY_ADD();      // CALL MY_ADD
}

```

Program 5.25(b) A simple example of subroutine that performs 16-Bit Addition, (without passing of parameters)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.2(b).

```

#include <stdint.h>
// Simulated working register and carry flag
uint8_t WREG;
uint8_t C = 0;

```



```

// Simulated File Select Registers (pointers to memory)
uint8_t *FSR0;
uint8_t *FSR1;
uint8_t *FSR2;

// MY_SR routine
void MY_SR(void)
{
    // MOVF POSTINC0, 0, 0
    WREG = *FSR0++;

    // ADDWFC POSTINC1, 0, 0
    WREG = WREG + *FSR1++ + C;
    C = (WREG < *FSR1) ? 1 : 0;           // approximate carry behavior (simplified)

    // MOVWF POSTINC2, 0
    *FSR2++ = WREG;
}

// START routine
void START(void)
{
    // LFSR initialization
    FSR0 = (uint8_t*)0x110;
    FSR1 = (uint8_t*)0x120;
    FSR2 = (uint8_t*)0x130;

    // Call routine twice
    MY_SR();
    MY_SR();

    // NOP (no operation)
}

```

Program 5.25(c) A simple example of subroutine that performs 8-Bit Addition, (passing of parameters using registers)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.1(b).

```
#include <xc.h> // Standard header for PIC18F458
// Simulate memory-mapped registers
volatile unsigned char memory_0x10; // Equivalent to address 0x10
volatile unsigned char memory_0x20; // Equivalent to address 0x20
volatile unsigned char memory_0x30; // Equivalent to address 0x30
volatile unsigned char TOCON; // Timer0 Control register

// Subroutine MY_ADD
void MY_ADD(unsigned char w)
{
    TOCON += w; // ADDWF TOCON, W
    memory_0x30 = TOCON; // MOVWF 0x30
}

int main()
{
    unsigned char w;

    // Main START routine
    w = memory_0x10; // MOVF 0x10, W
    TOCON = w; // MOVWF TOCON

    w = memory_0x20; // MOVF 0x20, W
    MY_ADD(w); // CALL MY_ADD

    while(1); // Infinite loop to prevent program from exiting
    return 0;
}
```

Program 5.25(d) A simple example of subroutine that performs 8-Bit Addition, (with passing of parameters in file register, accessed using direct addressing mode in subroutine)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.1(b). Here the file registers 0x10, 0x20, 0x30 are loaded with values using immediate addressing mode instructions in the main program and are accessed in subroutine using direct addressing mode.

```

#include <stdint.h>
// Map PIC memory locations to C variables
#define REG10 ((volatile uint8_t)0x10)
#define REG20 ((volatile uint8_t)0x20)
#define REG30 ((volatile uint8_t)0x30)
// Equivalent of MY_ADD subroutine
void MY_ADD(void)
{
    REG30 = REG10 + REG20;
}
// Main program (START)
void main(void)
{
    REG10 = 0x34;    // MOVLW 0x34 / MOVWF 0x10
    REG20 = 0x23;    // MOVLW 0x23 / MOVWF 0x20
    MY_ADD();        //                                CALL    MY_ADD
    while(1);        // typical embedded loop
}

```

Program 5.25(e) A simple example of subroutine that performs 8-Bit Addition, (with passing of parameters in file register, accessed using pointers - indirect/index addressing mode in subroutine)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.1(c). Here data is stored in file registers in main program using immediate addressing mode instructions and is later utilized through indirect addressing mode in subroutine. The pointer 1, 2 & 3 values of 0x110, 0x120, 0x130 are passed through physical registers FSR0, FSR1 & FSR2 and the respective addresses are accessed through non-physical registers POSTINC0, POSTINC1 AND POSTINC2 in the subroutine in indirect/indexed addressing mode.

```

#include <stdint.h>
int main()
{
    // Simulate memory (addresses 0x110, 0x120, 0x130)
    uint8_t memory[0x140] = {0};    // Make sure size covers all used addresses

    // Pointers equivalent to FSR0, FSR1, FSR2

```

```

uint8_t *FSR0;
uint8_t *FSR1;
uint8_t *FSR2;

// Initialize values
memory[0x110] = 0x34;
memory[0x120] = 0x23;

// Setup FSR pointers
FSR0 = &memory[0x110];
FSR1 = &memory[0x120];
FSR2 = &memory[0x130];

// Clear carry (simulate)
uint8_t carry = 0;

// Equivalent of MY_ADD
uint8_t WREG = *FSR0++;           // MOVF POSTINC0
uint16_t sum = WREG + *FSR1 + carry; // ADDWFC POSTINC1
WREG = sum & 0xFF;                // Keep only 8 bits
carry = (sum > 0xFF) ? 1 : 0;     // Update carry
*FSR2++ = WREG;                  // MOVWF POSTINC2

// memory[0x130] now contains 0x34 + 0x23 = 0x57
return 0;
}

```

Program 5.25(f) A simple example of subroutine that performs 8-Bit Addition, (with passing of parameters using variables – direct addressing)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.1(b). Here the variables VAR1, VAR2 and VAR3 created using Variable directive are used in place of file registers. The values passed into the subroutine are the values that are assigned before the subroutine definition and not before the subroutine call. To demonstrate this VAR1, VAR2 & VAR3 are assigned 3 different values in different places in the program file. 0x10, 0x20 and 0x30 are assigned before the definition of MY_ADD1 subroutine in line 2,3 and 4. 0x11, 0x21 and 0x31 are assigned after the definition of MY_ADD1 subroutine in line 12,13 and 14 before the

MY_ADD1 subroutine is called. 0x12, 0x22 and 0x32 are assigned before the definition of MY_ADD2 subroutine in line 19,20 and 21 after the MY_ADD2 subroutine is called but assigned before the definition of MY_ADD2 subroutine. The values assigned to the variables before the definition of the subroutine are the values considered and not the values assigned before the subroutine call.

```
#include <stdint.h>
// Global variables
uint8_t VAR1, VAR2, VAR3;

// Subroutine equivalent to MY_ADD1
void MY_ADD1(void)
{
    VAR3 = VAR1 + VAR2;
}

// Subroutine equivalent to MY_ADD2
void MY_ADD2(void)
{
    VAR3 = VAR1 + VAR2;
}

int main(void)
{
    // Initialization
    VAR1 = 0x11;
    VAR2 = 0x21;
    VAR3 = 0x31;

    // Call subroutines
    MY_ADD1();
    MY_ADD2();

    // Infinite loop (HERE: BRA HERE)
    while(1)
    {
        // Do nothing (NOP)
```

```

    }

    // Code below never executes due to infinite loop
    // VAR1 = 0x12;
    // VAR2 = 0x22;
    // VAR3 = 0x32;

    return 0;
}

```

Program 5.25(g) A simple example of subroutine that performs 8-Bit Addition, (with passing of parameters using SET directive – direct addressing)

The purpose of this program is to demonstrate the use of subroutine by modifying the simple example already discussed earlier in program 5.1(b). Here the variables VAR1, VAR2 and VAR3 created using set directive are used in place of file registers. The values passed into the subroutine are the values that are assigned before the subroutine definition and not before the subroutine call. The only difference is SET directive is used instead of a variable directive.

```

#include <xc.h>
#include <stdint.h>

// First memory block (used by MY_ADD1 before START reassigns)
volatile uint8_t VAR1 __at(0x10);
volatile uint8_t VAR2 __at(0x20);
volatile uint8_t VAR3 __at(0x30);

// Second memory block (used after START reassigns)
volatile uint8_t VAR1_2 __at(0x11);
volatile uint8_t VAR2_2 __at(0x21);
volatile uint8_t VAR3_2 __at(0x31);

void MY_ADD1(void)
{
    VAR3 = VAR1 + VAR2;
}

```

```

void MY_ADD2(void)
{
    VAR3_2 = VAR1_2 + VAR2_2;
}

void main(void)
{
    // First set of operations (before reassignment in assembly)
    VAR1 = 0;           // values would normally be set elsewhere
    VAR2 = 0;

    MY_ADD1();

    // Second set (equivalent to START reassigning addresses in assembly)
    VAR1_2 = 0;
    VAR2_2 = 0;

    MY_ADD2();

    while (1)
    {
        // HERE: infinite loop (BRA HERE)
    }
}

```

Program 5.26 Calculator without subroutines across banks

```

#include <stdint.h>
// Simulated PIC memory map (simplified)
uint8_t mem[256];
// Simulated 16-bit multiply result registers
uint8_t PRODL, PRODH;

void start(void)
{
    uint8_t W;

    // START

```

```

W = mem[0x00];

if (mem[0x01] == W)
{
    // Case 1
    mem[0x30] = mem[0x10] + mem[0x20];
    goto LEND;
}

L1:
if (mem[0x02] == W)
{
    // Case 2
    W = mem[0x50] - mem[0x40];
    mem[0x60] = W;
    goto LEND;
}

L2:
if (mem[0x03] != W)
{
    goto LEND;
}

// Case 3 (only when mem[0x03] == mem[0x00])
W = mem[0x70];
uint16_t result = (uint16_t)W * (uint16_t)mem[0x80];

PRODL = (uint8_t)(result & 0xFF);
PRODH = (uint8_t)(result >> 8);

mem[0x90] = PRODL;
mem[0x91] = PRODH;

LEND:
return;
}

```


Program 5.27 Calculator with subroutines across banks

```
#include <stdint.h>

// Memory-mapped variables
uint8_t var0 = 0; // 0x00
uint8_t var1 = 0; // 0x01
uint8_t var2 = 0; // 0x02
uint8_t var3 = 0; // 0x03

uint8_t val10 = 0; // 0x10
uint8_t val20 = 0; // 0x20
uint8_t result30 = 0; // 0x30

uint8_t val40 = 0; // 0x40
uint8_t val50 = 0; // 0x50
uint8_t result60 = 0; // 0x60

uint8_t val70 = 0; // 0x70
uint8_t val80 = 0; // 0x80
uint16_t result90 = 0; // 0x090/0x091 (16-bit)

void MY_ADD()
{
    result30 = val10 + val20;
}

void MY_SUB()
{
    result60 = val40 - val50;
}

void MY_MUL()
{
    result90 = (uint16_t)val70 * (uint16_t)val80;
}
```

```

void START()
{
    if (var0 == 1)
    {
        MY_ADD();
    }
    else if (var0 == 2)
    {
        MY_SUB();
    }
    else if (var0 == 3)
    {
        MY_MUL();
    }
    // else NOP
}

```

Program 5.28 A simple Fast Save based example – Modules: Call(S) & Return(S)

```

#include <p18f458.h>
#include <stdint.h>
/* Memory-mapped variables (same addresses as assembly) /
volatile uint8_t * const VAR20 = (uint8_t)0x20;
volatile uint8_t * const VAR21 = (uint8_t*)0x21;
volatile uint8_t * const VAR30 = (uint8_t*)0x30;
volatile uint8_t * const VAR31 = (uint8_t*)0x31;
volatile uint8_t * const VAR40 = (uint8_t*)0x40;
volatile uint8_t * const VAR41 = (uint8_t*)0x41;
/* Function equivalent of MY_ADD1 */
uint8_t MY_ADD1(void)
{
    uint8_t WREG;
    /*          MOVLW          0x18          */
    WREG          =          0x18;

    /*          ADDLW          0x28          */
    WREG = WREG + 0x28; //          WREG          =          0x40

```

```

/*          MOVWF          VAR2          (0x20)          */
*VAR20          =          WREG;

/*          MOVFF          STATUS,          VAR2          (0x21)          */
*VAR21          =          STATUS;

/*          RETURN          */
return WREG;
}

/* Main program equivalent of START */
void main(void)
{
    uint8_t WREG;
    /* MOVLW 0x34 / MOVWF VAR1 (0x30) */
    WREG = 0x34;
    *VAR30 = WREG;

    /* MOVFF STATUS, VAR1 (0x31) */
    *VAR31 = STATUS;

    /* CALL MY_ADD1 */
    WREG = MY_ADD1();

    /*          MOVWF          VAR1          (0x40)          */
    *VAR40 = WREG;

    /* MOVFF STATUS, VAR1 (0x41) */

    *VAR41          =          STATUS;

    /*          NOP          (do          nothing)          */
    while(1);
}

```

Program 5.29(a) Simple Marcos Example 1

```

#include <stdint.h>

#include <stdbool.h>

void program(void)
{
    // Simulated memory regions
    uint8_t *FSR0 = (uint8_t *)0x110;
    uint8_t *FSR1 = (uint8_t *)0x120;
    uint8_t *FSR2 = (uint8_t *)0x130;
    uint8_t carry = 0;
    for (int i = 0; i < 2; i++)
    {
        uint16_t temp;

        //          MOVF          POSTINC0          ->          W
        uint8_t      a          =          *FSR0++;
        //  ADDWFC  POSTINC1  ->  W  =  W  +  f  +  carry
        uint8_t b = *FSR1++;

        temp = (uint16_t)a + (uint16_t)b + carry;

        // update carry flag
        carry = (temp > 0xFF) ? 1 : 0;

        // MOVWF POSTINC2
        *FSR2++ = (uint8_t)temp;
    }

    // NOP (no operation)
}

```

Program 5.29(b) Marcos Example with parameter passing.

```

#include <stdint.h>

```

```

// Simulated PIC file registers (addresses 0x20–0x23)
volatile uint8_t REG_0x20;
volatile uint8_t REG_0x21;
volatile uint8_t REG_0x22;
volatile uint8_t REG_0x23;

int main(void)
{
    volatile uint8_t WREG; // working register
    __asm("NOP");

    WREG = 0x11 + REG_0x20;
    WREG = 0x22 + REG_0x21;
    WREG = 0x33 + REG_0x22;
    WREG = 0x44 + REG_0x23;

    __asm("NOP");

    while (1);
}

```

Program 5.30 Marcos with a Local Variable L1 for ‘Times’ number of loops

```

#include <xc.h> // or <p18f458.h> depending on compiler
// Simulate fixed RAM locations 0x20 and 0x21
volatile unsigned char loc1 @ 0x20;
volatile unsigned char loc2 @ 0x21;

// Equivalent of NOP instruction
#define NOP() asm("nop")

// Delay macro equivalent to MY_MDELAY
void MY_MDELAY(unsigned char times, volatile unsigned char *loc)
{
    *loc = times;

    while (*loc != 0)

```

```

        {
            NOP();
            NOP();
            (*loc)--;
        }
    }

void main(void)
{
    while (1)
    {
        asm("nop");    // START NOP
        MY_MDELAY(0x05, &loc1);
        MY_MDELAY(0x08, &loc2);
        asm("nop");
    }
}

```